YAZILIM MÜHENDİSLİĞİ · VİZE ÇALIŞMA NOTLARI

Bölüm 11
Architectural design

Mimari Tasarım

Kitap s.241–265 (Bölüm 11 tamamı)

Kaynak: Ian Sommerville, *Software Engineering*, 8. baskı
İçerik: Giriş · Sayfa sayfa açıklama, hoca anlatımı ve deftere not · Bölüm sonu: Terim sözlüğü, ⭐ Mutlaka bil, Sınav soruları, Mini test

G Giriş: Bu bölüm ne anlatıyor?

Bölüm 11, kitabın **Part 3: Design** kısmının ilk bölümü. Bu kısmın genel fikri şu: yazılım tasarımının özü, yazılımın **mantıksal organizasyonu** hakkında karar vermektir. Yazara göre tasarım **yaratıcı** bir süreçtir, herkese uyan bir "tarif" yoktur. Tasarım, mevcut tasarım örneklerine bakarak ve tasarımı başkalarıyla tartışarak öğrenilir. Bölüm 11–13 yazılımın soyut yapılarını anlatır, Bölüm 11 bunların ilkidir.

Bölüm 11 **yazılım mimarisi** ve **mimari tasarım** kavramlarını tanıtıyor. Beş kısımdan oluşuyor:

- **11.1 Architectural design decisions:** Mimari tasarımda verilen temel kararlar.
- **11.2 System organisation:** Repository, client-server ve layered (katmanlı) modeller.
- **11.3 Modular decomposition styles:** Nesne yönelimli ayrıştırma ve fonksiyon odaklı boru hattı (pipeline).
- **11.4 Control styles:** Merkezi kontrol (call-return, manager) ve olay güdümlü kontrol (broadcast, interrupt).
- **11.5 Reference architectures:** OSI modeli ve CASE referans mimarisi (ECMA).

Bölümün amaçları: (1) yazılımın mimari tasarımının neden önemli olduğunu anlamak, (2) mimari tasarım sürecinde sistem mimarisi hakkında verilmesi gereken kararları anlamak, (3) genel sistem organizasyonu, modüler ayrıştırma ve kontrolü kapsayan birbirini tamamlayan **üç mimari stil** grubunu tanımak, (4) referans mimarilerin mimari kavramları iletmek ve sistem mimarilerini değerlendirmek için nasıl kullanıldığını anlamak.

Sayfa 242–244 — Mimari tasarım nedir? · Avantajları · NFR'lere göre mimari · Blok diyagramları (Şekil 11.1)

📖 Sayfanın özeti

TANIM

Büyük sistemler her zaman ilişkili hizmetler sunan **alt sistemlere** ayrılır. Bu alt sistemleri belirleme ve alt sistemlerin **kontrolü ve iletişimi** için bir çerçeve kurma sürecine **mimari tasarım (architectural design)** denir. Bu sürecin çıktısı **yazılım mimarisinin** tanımıdır. Bölüm 4'teki modelde mimari tasarım, tasarım sürecinin **ilk aşamasıdır** ve tasarım ile gereksinim mühendisliği arasında **kritik bir bağdır**. Sistemin ana bileşenlerini ve aralarındaki iletişimi belirleyen temel yapısal çerçeveyi kurar.

MİMARİYİ AÇIKÇA TASARLAMANIN 3 AVANTAJI (BASS)

1) Stakeholder communication: mimari, sistemin farklı paydaşlar arasında tartışmaya odak olabilecek **üst seviye** bir sunumudur. **2) System analysis:** mimariyi erken aşamada açık hale getirmek analiz gerektirir. Mimari kararlar sistemin **performans, güvenilirlik ve bakım kolaylığı** gibi kritik gereksinimleri karşılayıp karşılamayacağını derinden etkiler. **3) Large-scale reuse:** mimari modeli, sistemin nasıl düzenlendiğinin ve bileşenlerin nasıl birlikte çalıştığının kısa, yönetilebilir bir tanımıdır. Benzer gereksinimli sistemlerin mimarisi çoğu zaman aynıdır, bu yüzden **büyük ölçekli yeniden kullanımı** destekleyebilir (aynı mimarinin ilişkili sistemlerde kullanıldığı ürün hattı mimarileri, Bölüm 18).

HOFMEİSTER'E GÖRE

Mimari tasarım aşaması tasarımcıları önemli tasarım konularını sürecin **erken** aşamasında düşünmeye zorlar. Yazılım mimarisi, sistem gereksinimlerini **müzakere etmek** ve müşteriler, geliştiriciler ve

yöneticilerle tartışmaları yapılandırmak için bir **tasarım planı** olarak kullanılabilir. Ayrıca **karmaşıklık yönetimi** için temel bir araçtır: ayrıntıları gizler ve tasarımcıların ana sistem soyutlamalarına odaklanmasını sağlar.

NFR'LER MİMARİYİ BELİRLER (BOSCH)

Sistem mimarisi performansı, sağlamlığı, dağıtılabilirliği ve bakım kolaylığını etkiler. Seçilen stil ve yapı bu yüzden **fonksiyonel olmayan gereksinimlere** bağlı olabilir: **1) Performance:** kritik işlemler **az sayıda alt sistemde** toplanmalı, alt sistemler arası iletişim mümkün olduğunca az olmalı. Bu, iletişimi azaltmak için ince taneli (fine-grain) değil **büyük taneli (large-grain)** bileşenler kullanmak demek olabilir. **2) Security: katmanlı** bir yapı kullanılmalı, en kritik varlıklar **en içteki** katmanlarda korunmalı ve bu katmanlara yüksek seviyede güvenlik doğrulaması uygulanmalı. **3) Safety:** emniyetle ilgili işlemlerin hepsi **tek bir alt sistemde** ya da az sayıda alt sistemde toplanmalı. Bu, emniyet doğrulamasının maliyetini ve sorunlarını azaltır ve ilgili koruma sistemleri sağlamayı mümkün kılar. **4) Availability: yedekli (redundant)** bileşenler içermeli ki sistem durdurulmadan bileşenler değiştirilip güncellenebilsin (hata toleranslı mimariler Bölüm 20). **5) Maintainability:** kolayca değiştirilebilen **ince taneli (fine-grain), kendi kendine yeten** bileşenler kullanılmalı. Veri üreticileri tüketicilerden ayrılmalı ve **paylaşılan veri yapılarından kaçınılmalı**.

ÇATIŞMA

Bu mimariler arasında çatışma olabilir: büyük taneli bileşenler **performansı**, ince taneli bileşenler **bakım kolaylığını** artırır. İkisi de önemliyse bir uzlaşma bulunmalıdır. Bu bazen sistemin **farklı kısımları için farklı mimari stiller** kullanarak sağlanabilir.

GEREKSİNİMLERLE ÖRTÜŞME

Gereksinim mühendisliği ile mimari tasarım arasında önemli bir örtüşme vardır. İdealde sistem tanımlaması tasarım bilgisi içermemeli, ama çok küçük sistemler dışında bu gerçekçi değildir. Tanımlamayı yapılandırmak ve düzenlemek için mimari ayrıştırma gerekir (ör. Bölüm 2'deki hava trafik kontrol mimarisi, alt sistem tanımlamasının başlangıç noktası olarak kullanılabilir).

BLOK DİYAGRAMLARI VE PAKETLEME ROBOTU (ŞEKİL 11.1)

Alt sistem tasarımı, sistemin her biri kendi başına büyük bir sistem olabilecek **büyük taneli bileşenlere** soyut olarak ayrılmasıdır. Genelde **blok diyagramlarıyla** anlatılır: her kutu bir alt sistemdir, kutu içindeki kutular alt sistemin kendisinin de alt sistemlere ayrıldığını gösterir, oklar veri ya da kontrol sinyallerinin ok yönünde geçtiğini gösterir. Blok diyagramları farklı disiplinlerden insanların kolayca anlayabileceği üst seviye bir resim sunar. Şekil 11.1 bir **paketleme robotu** sisteminin soyut mimarisidir: farklı türde nesneleri paketleyebilir. Bir **görüntü (vision)** alt sistemiyle konveyördeki nesneleri seçer, nesnenin türünü belirler (object identification) ve doğru paketleme türünü seçer (packaging selection). Sonra nesneleri teslim konveyöründen paketlenmek üzere alır ve paketlenmiş nesneleri başka bir konveyöre koyar. Alt sistemler: vision system, object identification system, arm controller, gripper controller, packaging selection system, packing system, conveyor controller.



Şekil 11.1 (sadeleştirilmiş): Paketleme robotu kontrol sisteminin alt sistemleri

KUTU-ÇİZGİ DİYAGRAMLARININ DEĞERİ

Bass ve arkadaşları basit kutu-çizgi diyagramlarının faydalı mimari gösterimler olmadığını savunur, çünkü bileşenler arası ilişkilerin doğasını ya da bileşenlerin dışarıdan görünen özelliklerini göstermezler. Yazara göre yazılım tasarımcısı açısından bu **tamamen doğrudur**. Ama bu tür model

paydaşlarla **iletişim** ve **proje planlaması** için etkilidir, çünkü ayrıntıyla dolu değildir. Paydaşlar sistemin soyut görünümünü anlayabilir. Model bağımsız geliştirilecek ana alt sistemleri gösterir, böylece yöneticiler bu sistemlerin geliştirilmesine insan atamaya başlayabilir. Kutu-çizgi diyagramları tek mimari gösterim olmamalı, ama birçok faydalı modelden biridir.

ALT SİSTEMLERE AYIRMA ZORDUR

Sistemi alt sistemlere nasıl ayıracağına karar vermek zor bir problemdir. Sistem gereksinimleri ana faktördür: gereksinimler ile alt sistemler arasında **yakın bir eşleşme** olan bir tasarım oluşturmaya çalışılmalıdır. Böylece gereksinimler değişirse değişiklik birkaç alt sisteme dağılmak yerine büyük ihtimalle **tek bir yerde (localised)** kalır. Bölüm 13'te alt sistem belirlemede başlangıç noktası olarak kullanılabilecek genel uygulama mimarileri anlatılıyor.

👩‍🏫 Hoca anlatımı

Mimari tasarım = sistemin **iskeletini** çizmek. Bir bina projesinde önce "kaç kat, hangi oda nerede, merdiven nerede, tesisat nereden geçecek" kararlarını verirsin, tuğlanın rengine sonra bakarsın. Yazılımda da önce "hangi büyük parçalar (alt sistemler) var ve birbirleriyle nasıl konuşacaklar" belirlenir.

3 avantaj (Bass): (1) **iletişim**, herkes aynı resme bakıp konuşur, (2) **analiz**, erken aşamada "bu mimari hızlı olur mu, güvenilir olur mu?" diye düşünürsün, (3) **yeniden kullanım**, benzer sistemler aynı mimariyi kullanabilir.

NFR'lere göre mimari bu bölümün en çok sorulan kısmı. Kısa hatırlatıcı:

Performance → **az ve büyük** parçalar (az iletişim, az bekleme). **Security** → **katmanlar**, değerli şeyler **en içte** (soğan gibi, ya da bankanın kasası gibi: dış kapı, iç kapı, kasa). **Safety** → emniyetle ilgili her şey **tek yerde** (kontrol etmesi kolay). **Availability** → **yedek parçalar** (biri bozulunca diğeri devreye girer, sistem durmaz). **Maintainability** → **küçük ve bağımsız** parçalar (LEGO gibi, kolayca değiştirilir), paylaşılan veri yok.

Çatışma: performans "büyük parça" der, bakım "küçük parça" der. Çözüm: sistemin farklı yerlerinde farklı stil kullan. Alıştırma 11.2 de böyle bir çatışmayı soruyor (availability ve security).

Blok diyagramı tartışması: Bass "kutu ve oklar yetersiz, ilişkinin doğasını göstermiyor" der. Yazar "doğru, ama yöneticiler ve müşterilerle konuşmak ve proje planlamak için çok iyi" der.

📝 Deftere not

- ⭐⭐ **Architectural design:** alt sistemleri belirleme ve alt sistemlerin **kontrolü ve iletişimi** için çerçeve kurma. Çıktı: **yazılım mimarisinin tanımı**.
- ⭐ Tasarım sürecinin **ilk aşaması**, tasarım ile RE arasında **kritik bağ**. Ana bileşenler + aralarındaki iletişim.
- ⭐⭐ **Mimariyi açıkça tasarlamanın 3 avantajı (Bass):**
  - Stakeholder communication: paydaşlar arası tartışmaya odak olan **üst seviye** sunum.
  - System analysis: erken analiz. Mimari kararlar **performans, güvenilirlik, bakım** gereksinimlerinin karşılanmasını etkiler.
  - Large-scale reuse: benzer sistemler aynı mimariyi kullanır (**product-line** mimarileri).

- **Hofmeister:** önemli tasarım konuları **erken** düşünülür. Mimari = gereksinimleri **müzakere** etmek için tasarım planı + **karmaşıklık yönetimi** aracı (ayrıntıyı gizler).
- ⭐⭐ **NFR'lere göre mimari (Bosch):**
  - **Performance:** kritik işlemler **az sayıda alt sistemde**, az iletişim, **large-grain** bileşenler.
  - **Security: katmanlı** yapı, kritik varlıklar **en iç** katmanda, yüksek güvenlik doğrulaması.
  - **Safety:** emniyet işlemleri **tek bir** ya da az sayıda alt sistemde (doğrulama ucuzlar, koruma sistemi eklenebilir).
  - **Availability: redundant** bileşenler, sistem durmadan değiştirme ve güncelleme.
  - **Maintainability: fine-grain, self-contained** bileşenler. Üretici ve tüketici ayrı, **paylaşılan veri yapısı yok**.
- ⭐ **Çatışma:** large-grain → performans, fine-grain → bakım. Çözüm: sistemin **farklı kısımlarında farklı stiller**.
- RE ve mimari tasarım **örtüşür**: spec'i yapılandırmak için mimari ayrıştırma gerekir (ör. ATC mimarisi).
- ⭐ **Alt sistem tasarımı: large-grain** bileşenlere soyut ayrıştırma. **Blok diyagramı:** kutu = alt sistem, kutu içinde kutu = alt sistemin alt sistemleri, ok = veri ya da kontrol sinyali yönü.
- **Şekil 11.1, paketleme robotu:** vision system, object identification, arm controller, gripper controller, packaging selection, packing system, conveyor controller.
- ⭐ **Bass:** kutu-çizgi diyagramları ilişkinin doğasını ve bileşenlerin dışarıdan görünen özelliklerini göstermez. **Yazar:** doğru, ama **paydaş iletişimi** ve **proje planlaması** için etkili (ayrıntısız, yöneticiler insan atayabilir). Tek gösterim olmamalı.
- ⭐ Alt sistemlere ayırma **zor**. Gereksinimler ile alt sistemler **yakın eşleşmeli**, böylece gereksinim değişikliği **tek yerde** (localised) kalır.

Sayfa 245–247 11.1 Architectural design decisions (Mimari tasarım kararları)

📖 Sayfanın özeti

KARAR BAKIŞ AÇISI

Mimari tasarım, fonksiyonel ve fonksiyonel olmayan gereksinimleri karşılayacak bir sistem organizasyonu kurmaya çalışılan **yaratıcı** bir süreçtir. Yaratıcı olduğu için süreçteki faaliyetler sistemin türüne, mimarın geçmişine ve deneyimine ve sistemin özel gereksinimlerine göre çok farklıdır. Bu yüzden süreci faaliyetler yerine **kararlar** açısından düşünmek daha faydalıdır. Mimarlar bilgi ve deneyimlerine dayanarak sistemi ve geliştirme sürecini derinden etkileyen şu **8 temel soruyu** cevaplamalıdır: (1) Tasarlanan sistem için şablon olabilecek **genel bir uygulama mimarisi** var mı? (2) Sistem birden çok işlemciye nasıl **dağıtılacak**? (3) Sistem için hangi **mimari stil(ler)** uygun? (4) Sistemi yapılandırmak için temel **yaklaşım** ne olacak? (5) Sistemdeki yapısal birimler **modüllere** nasıl ayrıştırılacak? (6) Birimlerin çalışmasını **kontrol** etmek için hangi strateji kullanılacak? (7) Mimari tasarım nasıl **değerlendirilecek**? (8) Sistemin mimarisi nasıl **belgelenecek**?

GENEL MİMARİLER VE DAĞITIM

Her yazılım sistemi benzersiz olsa da aynı uygulama alanındaki sistemler, alanın temel kavramlarını yansıtan **benzer mimarilere** sahiptir. Bunlar bilgi yönetim sistemleri mimarisi gibi oldukça genel ya da çok daha özel olabilir. Örneğin **uygulama ürün hatları**, belirli müşteri gereksinimlerini karşılayan

varyantlarıyla bir **çekirdek mimari** etrafında kurulan uygulamalardır. Mimariyi tasarlarken sistemin ve daha geniş uygulama sınıflarının ortak noktalarına ve bu mimarilerden ne kadar bilginin yeniden kullanılabileceğine karar verilir (Bölüm 13 ve 18). Gömülü sistemlerde ve kişisel bilgisayarlar için tasarlanan sistemlerde genelde tek işlemci vardır ve dağıtık mimari tasarlamak gerekmez. Ama büyük sistemlerin çoğu artık yazılımın birçok bilgisayara dağıtıldığı **dağıtık sistemlerdir**. Dağıtım mimarisinin seçimi sistemin **performansını ve güvenilirliğini** etkileyen temel bir karardır (Bölüm 12).

MİMARİ STİLLER

Bir yazılım sisteminin mimarisi belirli bir **mimari modele ya da stile** dayanabilir. Mimari stil, **client-server** ya da **katmanlı** mimari gibi bir **sistem organizasyonu kalıbıdır** (Garlan ve Shaw). Bu stillerin, uygulamalarının ve güçlü ve zayıf yanlarının farkında olmak önemlidir. Ama büyük sistemlerin çoğu tek bir stile uymaz: sistemin farklı kısımları farklı stillerle tasarlanabilir ve bazen genel mimari, farklı stillerin birleştirildiği **bileşik (composite)** bir mimaridir. Garlan ve Shaw'un stil kavramı sonraki üç tasarım sorusunu kapsar: gereksinimleri karşılayacak en uygun **yapı** (client-server ya da katmanlı gibi) seçilir, yapısal birimleri **modüllere ayırma** stratejisine karar verilir, son olarak **kontrol modellemede** alt sistemlerin çalışmasının nasıl kontrol edileceğine karar verilir. Bu üç konu Bölüm 11.2–11.4'te.

DEĞERLENDİRME VE BELGELEME

Mimari tasarımı değerlendirmek **zordur**, çünkü bir mimarinin gerçek testi, sistem kurulduktan sonra fonksiyonel ve fonksiyonel olmayan gereksinimleri ne kadar iyi karşıladığıdır. Ama bazı durumlarda tasarım **referans ya da genel mimari modellerle** karşılaştırılarak bir miktar değerlendirme yapılabilir (Bölüm 11.5 ve 13). Mimari tasarım sürecinin ürünü bir **mimari tasarım belgesidir**. Sistemin grafiksel gösterimlerini ve açıklayıcı metinleri içerebilir. Sistemin alt sistemlere nasıl ayrıldığını, benimsenen yaklaşımı ve her alt sistemin modüllere nasıl ayrıldığını anlatmalıdır.

5 MİMARİ MODEL TÜRÜ

Grafiksel modeller mimariye farklı bakış açıları sunar: **1) Static structural model:** ayrı birimler olarak geliştirilecek alt sistemleri ya da bileşenleri gösterir. **2) Dynamic process model:** sistemin çalışma zamanında **süreçlere** nasıl düzenlendiğini gösterir (statik modelden farklı olabilir). **3) Interface model:** her alt sistemin **genel arayüzü** üzerinden sunduğu hizmetleri tanımlar. **4) Relationship models:** alt sistemler arası **veri akışı** gibi ilişkileri gösterir. **5) Distribution model:** alt sistemlerin bilgisayarlara nasıl **dağıtılabileceğini** gösterir.

SAYFA 247: ADL'LER

Bazı araştırmacılar sistem mimarilerini tanımlamak için **mimari tanımlama dilleri (ADL, Architectural Description Languages)** kullanılmasını önermiştir. ADL'lerin temel öğeleri **bileşenler (components)** ve **bağlayıcılardır (connectors)**, ayrıca iyi biçimli mimariler için kurallar ve yönergeler içerirler. Ama bütün özel diller gibi ADL'ler de sadece **dil uzmanlarının** anlayabildiği, alan ve uygulama uzmanlarının erişemediği dillerdir. Bu yüzden pratik açıdan analizleri zordur. Yazara göre az sayıda uygulamada kullanılacaklar. **UML** gibi gayriresmi modeller ve gösterimler mimari tanım için **en yaygın** kullanılan gösterim olmaya devam edecek.

👨‍🏫 Hoca anlatımı

Mimari tasarım **yaratıcı** bir iş, bu yüzden "şu adımları izle" demek yerine "şu soruları cevapla" demek daha mantıklı. **8 soru**yu kendi cümlelerinle hatırla: **(1)** Hazır bir şablon var mı? **(2)** Kaç bilgisayara dağıtılacak? **(3)** Hangi stil? **(4)** Temel yapı yaklaşımı ne? **(5)** Modüllere nasıl bölünecek? **(6)** Kontrol nasıl olacak? **(7)** Nasıl değerlendirilecek? **(8)** Nasıl belgelenecek?

Mimari stil = bir "kalıp", hazır bir yapı tarzı. Mimarlıkta "Gotik", "modern" gibi stiller vardır. Yazılımda da "client-server", "katmanlı", "repository" gibi stiller vardır. Bu bölümün geri kalanı bu stilleri anlatıyor. Ama gerçek sistemler genelde **karışık** stil kullanır.

Bölümün yapısı burada belli oluyor, iyi kavra: **3 karar grubu = 3 stil grubu**: (1) **System organisation** (11.2): genel yapı (repository, client-server, layered). (2) **Modular decomposition** (11.3): alt sistemleri modüllere bölme (OO, pipeline). (3) **Control** (11.4): kim kimi çalıştırıyor (centralised, event-based).

5 mimari model: **static** (parçalar neler), **dynamic** (çalışırken süreçler), **interface** (hizmetler), **relationship** (veri akışı), **distribution** (hangi bilgisayarda).

ADL konusunda yazarın görüşü: çok teknik, sadece uzmanlar anlar, pek kullanılmayacak. **UML** en yaygın kalacak.

📝 Deftere not

- ⭐ Mimari tasarım = **yaratıcı** süreç, faaliyetler sistem türüne, mimarın deneyimine ve gereksinimlere göre değişir. Bu yüzden **kararlar** açısından düşünülür.
- ⭐⭐ **8 temel karar sorusu:**
  - 1. Şablon olabilecek **genel uygulama mimarisi** var mı?
  - 2. Sistem işlemcilere nasıl **dağıtılacak**?
  - 3. Hangi **mimari stil(ler)**?
  - 4. Sistemi yapılandırmanın temel **yaklaşımı**?
  - 5. Yapısal birimler **modüllere** nasıl ayrışacak?
  - 6. Birimlerin çalışmasını **kontrol** stratejisi?
  - 7. Mimari nasıl **değerlendirilecek**?
  - 8. Mimari nasıl **belgelenecek**?
- Aynı alandaki sistemler **benzer mimariler** taşır. **Product lines** = çekirdek mimari + müşteriye özel varyantlar (Bölüm 13, 18).
- Gömülü ve PC sistemleri: genelde tek işlemci. Büyük sistemler: **dağıtık**. Dağıtım mimarisi **performans ve güvenilirliği** etkiler (Bölüm 12).
- ⭐⭐ Architectural style (Garlan & Shaw): **sistem organizasyonu kalıbı** (ör. client-server, layered). Büyük sistemler genelde tek stile uymaz, **composite** mimari olabilir.
- ⭐ Stil kavramı **3 kararı** kapsar: **yapı** (11.2) → **modüllere ayrıştırma** (11.3) → **kontrol modeli** (11.4).
- ⭐ Değerlendirme **zor**: gerçek test, kurulumdan sonra gereksinimleri karşılama. Kısmen **referans ve genel mimarilerle karşılaştırma**.
- Ürün: **mimari tasarım belgesi** (grafikler + metin): alt sistemlere ayrılma, yaklaşım, modüllere ayrılma.
- ⭐⭐ **5 mimari model:**
  - **Static structural model:** ayrı geliştirilecek alt sistemler ve bileşenler.
  - **Dynamic process model:** çalışma zamanında **süreçler**.
  - **Interface model:** her alt sistemin genel arayüzündeki **hizmetler**.
  - **Relationship models:** alt sistemler arası ilişkiler (**veri akışı**).

- **Distribution model:** alt sistemlerin **bilgisayarlara dağılımı**.
- ⭐ **ADL (Architectural Description Languages):** temel öğeler **components + connectors** + kurallar. Sadece **dil uzmanları** anlar, pratikte analizi zor, az kullanılacak. **UML** gibi gayriresmi gösterimler **en yaygın**.

Sayfa 247–252 — 11.2 System organisation · Repository (Şekil 11.2) · Client-server (Şekil 11.3) · Layered (Şekil 11.4)

📖 Sayfanın özeti

SİSTEM ORGANİZASYONU

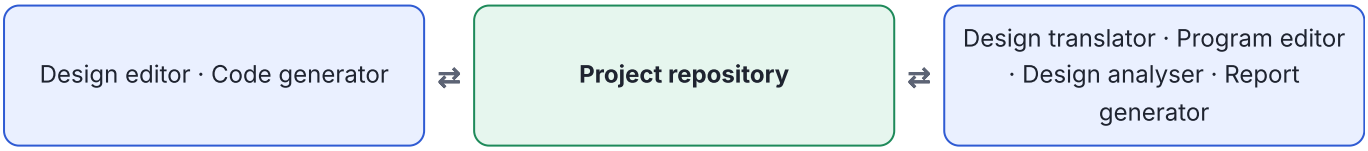
Sistem organizasyonu, sistemi yapılandırmak için kullanılan temel stratejiyi yansıtır ve mimari tasarım sürecinin **başında** kararlaştırılmalıdır. Alt sistem yapısına doğrudan yansıyabilir, ama alt sistem modeli çoğu zaman organizasyon modelinden daha fazla ayrıntı içerir. Bu bölümde çok yaygın **üç organizasyon stili** anlatılıyor: **paylaşılan veri deposu (repository)**, **paylaşılan hizmetler ve sunucular (client-server)** ve sistemin fonksiyonel katmanlar halinde düzenlendiği **soyut makine ya da katmanlı** stil. Bu stiller ayrı ayrı ya da **birlikte** kullanılabilir (ör. paylaşılan bir veri deposu etrafında düzenlenen bir sistem, verinin daha soyut bir görünümünü sunmak için üstüne katmanlar kurabilir).

11.2.1 REPOSİTORY MODEL

Alt sistemler etkili çalışmak için bilgi alışverişi yapmalıdır. Bunun **iki yolu** vardır: (1) bütün paylaşılan veri, bütün alt sistemlerin erişebildiği **merkezi bir veritabanında** tutulur. Paylaşılan veritabanına dayalı bir sisteme **repository model** denir. (2) Her alt sistem **kendi veritabanını** tutar, veri diğer alt sistemlerle **mesaj** göndererek değiş tokuş edilir. Büyük miktarda veri kullanan sistemlerin çoğu paylaşılan bir veritabanı etrafında düzenlenir. Bu model, verinin bir alt sistemde üretilip başka bir alt sistemde kullanıldığı uygulamalara uygundur. Örnekler: komuta-kontrol sistemleri, yönetim bilgi sistemleri, **CAD** sistemleri ve **CASE araç takımları**.

ŞEKİL 11.2: CASE ARAÇ TAKIMI

Paylaşılan bir **proje deposu (project repository)** etrafında: design editor, code generator, design translator, program editor, design analyser ve report generator. CASE araçları için ilk paylaşılan depo muhtemelen 1970'lerin başında İngiliz **ICL** şirketi tarafından işletim sistemi geliştirmeyi desteklemek için yapıldı. Model, **Buxton**'ın Ada ile yazılan sistemlerin geliştirilmesini destekleyen **Stoneman** ortamı önerisiyle daha yaygın tanındı. O zamandan beri birçok CASE araç takımı paylaşılan depo etrafında geliştirildi.



Şekil 11.2: Repository tabanlı CASE araç takımı mimarisi

REPOSİTORY: AVANTAJ VE DEZAVANTAJLAR (4 ÇİFT)

1) + Büyük miktarda veriyi paylaşmanın **verimli** yoludur, veriyi alt sistemler arasında açıkça aktarmaya gerek yoktur. **–** Alt sistemler deponun **veri modeli üzerinde anlaşmak** zorundadır. Bu kaçınılmaz olarak her aracın özel ihtiyaçları arasında bir uzlaşmadır ve performansı olumsuz etkileyebilir. Veri modeli anlaşılan şemaya uymayan yeni alt sistemleri entegre etmek zor ya da imkânsız olabilir. **2) +** Veri üreten alt sistemler o verinin diğer alt sistemlerce nasıl kullanıldığıyla **ilgilenmek zorunda değildir**. **–** Anlaşılmış bir veri modeline göre büyük miktarda bilgi üretildiği için

evrim zor olabilir. Bunu yeni bir modele çevirmek kesinlikle pahalı, zor ya da imkânsız olabilir. **3) +** Yedekleme, güvenlik, erişim kontrolü ve hatadan kurtarma gibi faaliyetler **merkezileşir**. Bunlar depo yöneticisinin sorumluluğudur, araçlar asıl işlevlerine odaklanabilir. **–** Farklı alt sistemlerin güvenlik, kurtarma ve yedekleme için **farklı ihtiyaçları** olabilir, ama repository modeli bütün alt sistemlere **aynı politikayı** dayatır. **4) +** Paylaşım modeli depo şeması üzerinden görünürdür. Anlaşılmış veri modeline uyumlu oldukları sürece **yeni araçları entegre etmek** kolaydır. **–** Depoyu birden çok makineye **dağıtmak zor** olabilir. Mantıksal olarak merkezi bir depoyu dağıtmak mümkün olsa da veri tekrarı ve tutarsızlık sorunları olabilir.

BLACKBOARD MODELİ

Yukarıdaki modelde depo **pasiftir**, kontrol depoyu kullanan alt sistemlerin sorumluluğundadır. Alternatif olarak yapay zekâ sistemlerinden türeyen **"blackboard" (kara tahta)** modeli vardır: belirli veriler hazır olunca alt sistemleri **tetikler**. Depodaki verinin biçimi daha az yapılandırılmışsa uygundur, çünkü hangi aracın çalıştırılacağına ancak veri analiz edildikten sonra karar verilebilir (Nii, Bosch).

11.2.2 CLİENT-SERVER MODEL

Sistemin bir **hizmetler** kümesi, ilgili **sunucular** ve bu hizmetlere erişip kullanan **istemciler** olarak düzenlendiği modeldir. **3 ana bileşen:** (1) diğer alt sistemlere hizmet sunan **sunucular** kümesi. Ör: yazdırma hizmeti sunan print server, dosya yönetim hizmeti sunan file server, derleme hizmeti sunan compile server. (2) Sunucuların hizmetlerini çağıran **istemciler** kümesi. Bunlar genelde kendi başına alt sistemlerdir ve bir istemci programın birçok örneği eşzamanlı çalışabilir. (3) İstemcilerin bu hizmetlere erişmesini sağlayan bir **ağ**. Bu kesin şart değildir (istemci ve sunucu tek makinede çalışabilir), ama pratikte çoğu client-server sistem dağıtık olarak gerçekleştirilir. İstemcilerin sunucuların ve sundukları hizmetlerin adlarını bilmesi gerekebilir, ama sunucuların istemcilerin kimliğini ya da sayısını bilmesi gerekmez. İstemciler hizmetlere **uzak prosedür çağrılarıyla** ya da WWW'deki **http** gibi bir **istek-cevap (request-reply)** protokolüyle erişir: istemci sunucuya istek gönderir ve cevap gelene kadar bekler.

ŞEKİL 11.3: FİLM VE FOTOĞRAF KÜTÜPHANESİ

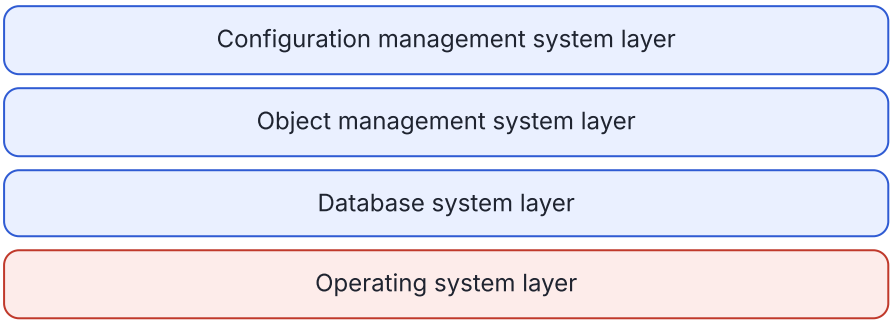
Çok kullanıcılı, web tabanlı bir film ve fotoğraf kütüphanesi sistemi. İstemciler (client 1–4) internet üzerinden şu sunuculara erişir: **katalog sunucusu** (kütüphane kataloğu), **video sunucusu** (film klipleri), **resim sunucusu** (dijital fotoğraflar) ve **web sunucusu** (film ve fotoğraf bilgisi). Farklı medya türlerini farklı sunucular yönetir. Video kareleri hızlı ve senkron iletilmeli ama nispeten düşük çözünürlükte olabilir, sıkıştırılmış saklanabilir, bu yüzden video sunucusu farklı formatlara sıkıştırma ve açma işini yapabilir. Fotoğraflar ise **yüksek çözünürlükte** tutulmalıdır, bu yüzden ayrı bir sunucuda olmaları uygundur. Katalog çeşitli sorguları yanıtlamalı ve film ve video bilgisi içeren web bilgi sistemine ve film ve video klip satışını destekleyen bir e-ticaret sistemine bağlantılar vermelidir. İstemci programı bu hizmetlere web tarayıcısıyla kurulmuş entegre bir kullanıcı arayüzüdür.

CLİENT-SERVER: AVANTAJ VE DEZAVANTAJ

En önemli avantajı dağıtık bir mimari olmasıdır. Birçok dağıtık işlemcili ağ bağlantılı sistemler etkili kullanılabilir. Yeni bir sunucu eklemek ve sistemin geri kalanıyla entegre etmek ya da sunucuları sistemin diğer kısımlarını etkilemeden **şeffaf şekilde yükseltmek** kolaydır (Bölüm 12). **Dezavantajı:** yeni bir sunucunun bütün faydasını almak için mevcut istemci ve sunucularda değişiklik gerekebilir. Sunucular arasında **paylaşılan bir veri modeli olmayabilir** ve alt sistemler verilerini farklı şekillerde düzenleyebilir. Bu, performansı optimize etmek için her sunucuda özel veri modelleri kurulabileceği anlamına gelir. XML tabanlı veri gösterimi kullanılırsa bir şemadan diğerine dönüştürmek nispeten kolay olabilir, ama XML veriyi göstermenin **verimsiz** bir yoludur ve performans sorunları çıkabilir.

11.2.3 LAYERED MODEL

Katmanlı model (bazen **abstract machine model** denir) sistemi her biri bir hizmet kümesi sunan **katmanlar** halinde düzenler. Her katman, makine dili o katmanın sunduğu hizmetlerle tanımlanan bir **soyut makine** olarak düşünülebilir. Bu "dil" bir sonraki soyut makine seviyesini gerçekleştirmek için kullanılır. Ör: bir dili gerçekleştirmenin yaygın yolu ideal bir "dil makinesi" tanımlayıp dili bu makinenin koduna derlemektir. Sonra bir çeviri adımı bu soyut makine kodunu gerçek makine koduna çevirir. Katmanlı modele örnekler: ağ protokollerinin **OSI referans modeli** (11.5) ve **Buxton**'ın Ada programlama destek ortamı (**APSE**) için önerdiği üç katmanlı model.



Şekil 11.4: Sürüm yönetim sisteminin katmanlı modeli (APSE yapısını yansıtır)

ŞEKİL 11.4 AÇIKLAMASI

En üstteki **konfigürasyon yönetim sistemi** nesnelerin sürümlerini yönetir ve genel konfigürasyon yönetimi olanakları sağlar (Bölüm 29). Bunu desteklemek için konfigürasyon öğeleri ya da nesneler için bilgi saklama ve yönetim hizmetleri sunan bir **nesne yönetim sistemi** kullanır. Bu sistem, temel veri saklama ve işlem yönetimi, geri alma (rollback), kurtarma ve erişim kontrolü gibi hizmetler sunan bir **veritabanı sistemi** üzerine kurulmuştur. Veritabanı yönetimi de gerçekleştirmesinde alttaki **işletim sistemi** olanaklarını ve dosya sistemini kullanır (diğer örnekler Bölüm 13'te).

KATMANLI MODEL: AVANTAJLAR

Katmanlı yaklaşım sistemlerin **artımlı geliştirilmesini** destekler: bir katman geliştirilince sunduğu hizmetlerin bir kısmı kullanıcılara açılabilir. Bu mimari **değiştirilebilir ve taşınabilirdir**. Arayüzü değişmediği sürece bir katman **eşdeğer başka bir katmanla değiştirilebilir**. Ayrıca katman arayüzleri değiştiğinde ya da bir katmana yeni olanaklar eklendiğinde sadece **komşu katman** etkilenir. Katmanlı sistemler makineye bağımlılıkları iç katmanlarda topladığı için uygulamanın **çok platformlu** gerçekleştirmelerini sağlamayı kolaylaştırır: farklı bir işletim sistemi ya da veritabanına uyum için sadece iç, makineye bağımlı katmanların yeniden gerçekleştirilmesi gerekir.

SAYFA 252: DEZAVANTAJLAR

Sistemi bu şekilde yapılandırmak **zor** olabilir. İç katmanlar her seviyede gereken dosya yönetimi gibi temel olanaklar sağlayabilir. Üst seviyedeki bir kullanıcının ihtiyaç duyduğu hizmetler bu yüzden birkaç seviye aşağıdaki hizmetlere erişmek için komşu katmanları **"delip geçmek" (punch through)** zorunda kalabilir. Bu modeli bozar, çünkü dış katman artık sadece hemen altındakine bağlı değildir. **Performans** da sorun olabilir, çünkü bazen birden çok seviyede komut yorumlanması gerekir. Çok katman varsa üst katmandan gelen bir hizmet isteği işlenmeden önce farklı katmanlarda birkaç kez yorumlanmak zorunda kalabilir. Bu sorunlardan kaçınmak için uygulamalar komşu katmanın hizmetlerini kullanmak yerine doğrudan iç katmanlarla iletişim kurmak zorunda kalabilir.

👨‍🏫 Hoca anlatımı

Bu bölümün **en önemli kısmı** bu 3 model! Sınavda neredeyse kesin çıkar. Her birinin **tanımını, örneğini, artısını ve eksisini** bil.

1) Repository (depo) modeli = herkesin kullandığı **ortak bir dolap**. Bir evde herkesin yemeklerini koyduğu ortak bir buzdolabı gibi. Kimse kimseye yemek taşımaz, herkes buzdolabına koyar ve alır. **Artı:** veri taşımaya gerek yok, yedekleme ve güvenlik tek merkezden, yeni araç eklemek kolay. **Eksi:** herkes buzdolabının düzenine uymak zorunda (ortak veri modeli), düzeni değiştirmek çok zor (evrim), herkese aynı kurallar (aynı yedekleme ve güvenlik politikası), buzdolabını birden çok eve dağıtmak zor. Örnek: **CASE araç takımları**.

Blackboard farkı: normal depo **pasif** (sadece bekler). Blackboard **aktif**: tahtaya yeni bir şey yazılınca ilgili uzmanı çağırır.

2) Client-server = **restoran** modeli. Garsonlar ve mutfaklar (sunucular) hizmet verir, müşteriler (istemciler) sipariş verir. Mutfak kimin sipariş verdiğini bilmek zorunda değil. Müşteri istek gönderir ve cevabı bekler (**request-reply**, ör. http). **3 bileşen:** sunucular, istemciler, ağ. **Artı: dağıtık**, yeni sunucu eklemek ve yükseltmek kolay. **Eksi:** sunucular arasında **ortak veri modeli yok**, değişiklik istemcileri de etkileyebilir. Örnek: **film ve fotoğraf kütüphanesi** (her medya türü kendi sunucusunda, çünkü video hızlı ve düşük çözünürlük, fotoğraf yüksek çözünürlük ister).

3) Layered (katmanlı) model = **apartman katları** ya da soğan. Her katman sadece **hemen altındakini** kullanır. Örnek: konfigürasyon yönetimi → nesne yönetimi → veritabanı → işletim sistemi. Bir başka ünlü örnek **OSI** ağ modeli. **Artı:** artımlı geliştirme, bir katman aynı arayüzle başka bir katmanla değiştirilebilir, değişiklik sadece komşuyu etkiler, **taşınabilir** (makineye bağlı şeyler iç katmanda, başka platforma geçerken sadece onu değiştir). **Eksi:** yapılandırmak zor (bazen üst katman alt katmanlara "delip geçmek" zorunda kalır), **performans** (istek her katmanda tekrar yorumlanır).

Alıştırma 11.3 tam olarak bu 3 modelin **artı ve eksilerini tablo halinde** istiyor. Bölüm sonunda tablo halinde hazırladım.

📝 Deftere not

- ⭐ **System organisation:** sistemi yapılandırmanın temel stratejisi, mimari tasarımın **başında** karar verilir.
- ⭐⭐ **3 organizasyon stili: Repository** (paylaşılan veri deposu) · **Client-server** (paylaşılan hizmetler ve sunucular) · **Layered / abstract machine** (katmanlar). Ayrı ya da **birlikte** kullanılabilir.
- ⭐ Alt sistemler arası veri paylaşımının **2 yolu:** (1) **merkezi veritabanı** = **repository model**, (2) her alt sistem **kendi DB'si** + **mesajlaşma**.
- ⭐ **Repository**, verinin bir alt sistemde üretilip başkasında kullanıldığı sistemlere uygun. Ör: komuta-kontrol, yönetim bilgi sistemleri, **CAD**, **CASE araç takımları**.
- **Şekil 11.2:** project repository + design editor, code generator, design translator, program editor, design analyser, report generator. İlk CASE deposu: **ICL** (1970'ler). **Buxton**'ın **Stoneman** (Ada) ortamı.
- ⭐⭐ **Repository artı ve eksileri:**
  - (+) Büyük veriyi paylaşmak **verimli**, açık aktarım yok · (–) alt sistemler **ortak veri modeline** uymalı (uzlaşma, performans düşebilir, uymayan yeni alt sistemi entegre etmek zor).
  - (+) Veri üretenler verinin **nasıl kullanıldığıyla ilgilenmez** · (–) **evrim zor** (büyük veriyi yeni modele çevirmek pahalı ya da imkânsız).

    - (+) Yedekleme, güvenlik, erişim kontrolü, kurtarma **merkezi** (depo yöneticisinin işi) · (–) alt sistemlerin farklı ihtiyaçlarına rağmen **aynı politika** dayatılır.
    - (+) Paylaşım şema üzerinden görünür, uyumlu **yeni araç eklemek kolay** · (–) depoyu birden çok makineye **dağıtmak zor** (veri tekrarı, tutarsızlık).
- ⭐ Normal depo **pasif** (kontrol alt sistemlerde). Blackboard model (yapay zekâdan): belirli veri hazır olunca alt sistemleri **tetikler**, verinin yapısı zayıfsa uygun.
- ⭐⭐ Client-server model: sistem = **hizmetler + sunucular + istemciler**. **3 bileşen:**
    - **Servers:** hizmet sunar (print server, file server, compile server).
    - **Clients:** hizmetleri çağırır, genelde kendi başına alt sistem, birden çok örnek eşzamanlı çalışabilir.
    - **Network:** erişimi sağlar (şart değil ama pratikte dağıtık).
- İstemci sunucu adlarını bilmeli. **Sunucu istemcilerin kimliğini ya da sayısını bilmez**. Erişim: **remote procedure call** ya da **request-reply** protokolü (**http**). İstemci istek gönderir ve **cevabı bekler**.
- ⭐ **Şekil 11.3, film ve fotoğraf kütüphanesi:** istemciler internet üzerinden catalogue, video, picture ve web sunucularına bağlanır. Video: hızlı, senkron, düşük çözünürlük, sıkıştırma. Fotoğraf: **yüksek çözünürlük**, ayrı sunucu. İstemci = web tarayıcısıyla entegre arayüz.
- ⭐⭐ **Client-server artı: dağıtık** mimari, ağdaki işlemcileri etkili kullanır, **yeni sunucu eklemek** ve sunucuları **şeffaf yükseltmek** kolay.
- ⭐⭐ **Client-server eksi:** yeni sunucunun faydası için istemci ve sunucularda değişiklik gerekebilir · sunucular arası **ortak veri modeli yok** (her sunucu kendi modelini kurabilir, performans için iyi) · **XML** dönüşümü kolaylaştırır ama **verimsizdir**, performans sorunu.
- ⭐⭐ Layered model (**abstract machine model**): sistem, her biri hizmet sunan **katmanlar**. Her katman = **soyut makine**, "dili" bir sonraki katmanı gerçekleştirmek için kullanılır. Ör: **OSI** modeli, Buxton'ın **APSE** 3 katmanı.
- ⭐ **Şekil 11.4 (sürüm yönetim sistemi):** Configuration management layer → Object management layer → Database system layer → Operating system layer.
- ⭐⭐ **Layered artıları: artımlı geliştirme** (katman bitince hizmetleri açılabilir) · **değiştirilebilir ve taşınabilir** (arayüz aynıysa katman **başkasıyla değiştirilebilir**) · değişiklik sadece **komşu katmanı** etkiler · makine bağımlılıkları **iç katmanda**, bu yüzden **çok platform** kolay.
- ⭐⭐ **Layered eksileri:** yapılandırmak **zor** (iç katmanların temel hizmetleri her seviyede gerekir, üst katman komşuları **delip geçer (punch through)**, model bozulur) · **performans** (istek birden çok katmanda yorumlanır). Çözüm: doğrudan iç katmanlarla iletişim.

Sayfa 252–255 — 11.3 Modular decomposition styles · Object-oriented (Şekil 11.5) · Function-oriented pipelining (Şekil 11.6)

📖 Sayfanın özeti

ALT SİSTEM VE MODÜL

Genel sistem organizasyonu seçildikten sonra alt sistemlerin **modüllere ayrıştırılması** yaklaşımına karar verilir. Sistem organizasyonu ile modüler ayrıştırma arasında katı bir ayrım yoktur (11.2'deki stiller bu seviyede de uygulanabilir), ama modüllerdeki bileşenler genelde alt sistemlerden **daha**

küçüktür, bu da başka ayrıştırma stillerine izin verir. Alt sistem ve modül arasında net bir ayrım yoktur, ama yazar şöyle düşünmeyi faydalı buluyor: **1) Alt sistem (sub-system)**, çalışması diğer alt sistemlerin sunduğu hizmetlere **bağlı olmayan**, kendi başına bir sistemdir. Alt sistemler modüllerden oluşur ve diğer alt sistemlerle iletişim için tanımlı **arayüzleri** vardır. **2) Modül (module)**, normalde diğer modüllere bir ya da daha fazla hizmet sunan bir sistem bileşenidir. Diğer modüllerin hizmetlerini kullanır. Normalde **bağımsız bir sistem sayılmaz**. Modüller genelde birkaç daha basit bileşenden oluşur.

2 ANA STRATEJİ

Bir alt sistemi modüllere ayrıştırmanın iki ana stratejisi vardır: **1) Object-oriented decomposition:** sistem **birbiriyle iletişim kuran nesneler** kümesine ayrıştırılır. **2) Function-oriented pipelining:** sistem, girdi verisini alıp çıktı verisine **dönüştüren** fonksiyonel modüllere ayrıştırılır. Nesne yönelimli yaklaşımda modüller, özel (private) durumları ve bu durum üzerinde tanımlı işlemleri olan **nesnelerdir**. Boru hattı modelinde modüller **fonksiyonel dönüşümlerdir**. İki durumda da modüller **sıralı bileşenler** ya da **süreçler** olarak gerçekleştirilebilir.

EŞZAMANLILIĞA ERKEN BAĞLANMA

Sistemde **eşzamanlılığa (concurrency) erken bağlanmaktan** kaçınılmalıdır. Eşzamanlı bir sistem tasarımından kaçınmanın avantajı, sıralı programların paralel sistemlerden daha kolay tasarlanması, gerçekleştirilmesi, doğrulanması ve test edilmesidir. Süreçler arası zaman bağımlılıklarını resmileştirmek, kontrol etmek ve doğrulamak zordur. En iyisi sistemleri modüllere ayrıştırmak, sonra **gerçekleştirme sırasında** bunların sıralı mı paralel mi çalışacağına karar vermektir.

11.3.1 NESNE YÖNELİMLİ AYRIŞTIRMA (ŞEKİL 11.5)

Nesne yönelimli bir mimari model, sistemi iyi tanımlanmış arayüzleri olan **gevşek bağlı (loosely coupled)** nesneler kümesi olarak yapılandırır. Nesneler diğer nesnelerin sunduğu hizmetleri çağırır (nesne modelleri Bölüm 8, OO tasarım Bölüm 14). Şekil 11.5, bir **fatura işleme sistemi** için nesne yönelimli mimari modeldir: sistem müşterilere fatura kesebilir, ödemeleri alabilir, bu ödemeler için makbuz ve ödenmemiş faturalar için hatırlatma gönderebilir. UML gösteriminde nesne sınıflarının adları ve öznitelikleri vardır, işlemler (varsa) dikdörtgenin alt kısmında tanımlanır. **Kesikli oklar** bir nesnenin başka bir nesnenin özniteliklerini ya da hizmetlerini kullandığını gösterir. Sınıflar: **Customer** (müşteri no, ad, adres, kredi süresi), **Payment** (fatura no, tarih, tutar, müşteri no), **Invoice** (fatura no, tarih, tutar, müşteri; işlemler: issue, sendReminder, acceptPayment, sendReceipt), **Receipt** (fatura no, tarih, tutar, müşteri no).

AÇIKLAMA

Nesne yönelimli ayrıştırma nesne sınıfları, öznitelikleri ve işlemleriyle ilgilenir. Gerçekleştirildiğinde bu sınıflardan nesneler oluşturulur ve nesne işlemlerini koordine etmek için bir kontrol modeli kullanılır. Bu örnekte **Invoice** sınıfının sistem işlevselliğini gerçekleştiren çeşitli işlemleri vardır. Bu sınıf müşterileri, ödemeleri ve makbuzları temsil eden diğer sınıfları kullanır.

OO: AVANTAJ VE DEZAVANTAJ

Avantajlar iyi bilinir: nesneler **gevşek bağlı** olduğu için bir nesnenin gerçekleştirmesi diğer nesneleri etkilemeden **değiştirilebilir**. Nesneler çoğu zaman **gerçek dünya varlıklarının** temsilleridir, bu yüzden sistemin yapısı kolay **anlaşılır**. Bu gerçek dünya varlıkları farklı sistemlerde kullanıldığı için nesneler **yeniden kullanılabilir**. Mimari bileşenlerin doğrudan gerçekleştirilmesini sağlayan nesne yönelimli programlama dilleri vardır. **Dezavantajlar:** hizmetleri kullanmak için nesneler diğer nesnelerin **adına ve arayüzüne açıkça başvurmak** zorundadır. Önerilen sistem değişikliklerini karşılamak için bir arayüz değişikliği gerekirse, değişen nesnenin **bütün kullanıcıları** üzerindeki etkisi

değerlendirilmelidir. Nesneler küçük ölçekli gerçek dünya varlıklarıyla temiz eşleşebilir, ama **daha karmaşık varlıkları** nesne olarak temsil etmek bazen zordur.

11.3.2 FONKSİYON ODAKLI BORU HATTI

Fonksiyon odaklı boru hattı ya da **veri akışı (data-flow)** modelinde fonksiyonel dönüşümler girdilerini işleyip çıktı üretir. Veri birinden diğerine akar ve sıra boyunca ilerlerken dönüşür. Her işleme adımı bir **dönüşüm (transform)** olarak gerçekleştirilir. Girdi verisi bu dönüşümlerden geçerek çıktıya dönüşür. Dönüşümler **sıralı ya da paralel** çalışabilir. Veri her dönüşüm tarafından **öğe öğe** ya da **tek bir toplu (batch)** halinde işlenebilir. Dönüşümler ayrı süreçler olarak gösterildiğinde bu modele, **Unix** sistemindeki terminolojiden dolayı bazen **pipe and filter** stili denir. Unix, veri kanalı görevi gören **borular (pipes)** ve fonksiyonel dönüşüm olan bir komutlar kümesi sunar. Bu modele uyan sistemler Unix komutlarını borular ve Unix kabuğunun kontrol olanaklarıyla birleştirerek gerçekleştirilebilir. **"Filter"** terimi kullanılır, çünkü bir dönüşüm girdi veri akışından işleyebileceği veriyi **"süzer"**.

BATCH SEQUENTİAL

Bu boru hattı modelinin varyantları bilgisayarlar otomatik veri işleme için ilk kullanıldığından beri kullanılmaktadır. Dönüşümler **sıralı** ise ve veri **toplu (batch)** işleniyorsa bu mimari modele **batch sequential** model denir. Bölüm 13'te anlatıldığı gibi **faturalama sistemleri** gibi veri işleme sistemleri için yaygın bir mimaridir. Veri işleme sistemleri genelde çok sayıda girdi kaydı üzerinde basit hesaplamalardan türeyen birçok çıktı raporu üretir.

ŞEKİL 11.6: FATURA SİSTEMİ BORU HATTI

Bir kurum müşterilere fatura kesmiştir. Haftada bir, yapılan ödemeler faturalarla **eşleştirilir**. Ödenmiş faturalar için **makbuz** kesilir, izin verilen ödeme süresi içinde ödenmemiş faturalar için **hatırlatma** gönderilir. Akış: Invoices → Read issued invoices → Identify payments (Payments girdisiyle) → (Issue receipts → Receipts) ve (Find payments due → Issue payment reminder → Reminders). Bu, fatura sisteminin sadece bir kısmının modelidir (fatura kesmek için başka dönüşümler gerekir). Nesne yönelimli eşdeğerinden farkına dikkat: **nesne modeli daha soyuttur**, çünkü işlemlerin **sırası** hakkında bilgi içermez.



Şekil 11.6: Fatura işleme sisteminin boru hattı modeli

PİPELİNE AVANTAJLARI

1) Dönüşümlerin **yeniden kullanımını** destekler. **2) Sezgiseldir**: birçok insan işini girdi ve çıktı işleme olarak düşünür. **3)** Yeni dönüşümler ekleyerek sistemi **geliştirmek** genelde kolaydır. **4)** Eşzamanlı ya da sıralı sistem olarak gerçekleştirmesi **basittir**.

PİPELİNE DEZAVANTAJLARI

Bu stilin ana sorunu, bütün dönüşümlerin tanıyabileceği **ortak bir veri aktarım formatı** olması gerekliliğidir. Her dönüşüm ya işleyeceği verinin formatı konusunda iletişim kurduğu dönüşümlerle anlaşmalı ya da iletilen bütün veri için standart bir format dayatılmalıdır. Dönüşümler bağımsız ve yeniden kullanılabilir olduğunda tek uygulanabilir yol ikincisidir. Unix'te standart format basitçe bir **karakter dizisidir**. Her dönüşüm girdisini ayrıştırmalı (parse) ve çıktısını anlaşılmış biçime çevirmelidir (unparse). Bu sistem yükünü artırır ve **uyumsuz veri formatları** kullanan dönüşümleri entegre etmenin imkânsız olması anlamına gelebilir. Ayrıca **etkileşimli sistemleri** boru hattı modeliyle yazmak zordur, çünkü işlenecek bir veri akışı gerekir. Basit metin girdi ve çıktısı bu şekilde modellenebilir, ama grafik kullanıcı arayüzlerinin fare tıklamaları ya da menü seçimleri gibi **olaylara** dayanan daha

karmaşık G/Ç formatları ve kontrolü vardır. Bunu boru hattı modeline uyumlu bir forma çevirmek zordur.

👩‍🏫 Hoca anlatımı

Önce **alt sistem ve modül farkı**: **Alt sistem** kendi başına çalışabilir, başkasına muhtaç değil (bir arabanın motoru gibi). **Modül** ise tek başına bir sistem sayılmaz, başka modüllerle birlikte çalışır (motordaki bir piston gibi).

Alt sistemi modüllere bölmenin **2 yolu**:

1) Object-oriented = sistemi **"şeylere"** böl: Müşteri, Fatura, Ödeme, Makbuz. Her nesnenin kendi bilgisi (öznitelik) ve yapabildikleri (işlem) var. **Artı:** gevşek bağlı (birini değiştir, diğerleri etkilenmez), gerçek dünyaya benzer (anlaşılır), yeniden kullanılabilir. **Eksi:** nesneler birbirinin **adını ve arayüzünü bilmek** zorunda (arayüz değişirse kullanan herkes etkilenir), karmaşık varlıkları nesneyle göstermek zor.

2) Function-oriented pipelining = sistemi **"işlemlere"** böl, bir **fabrika üretim bandı** gibi: hammadde girer, her istasyon bir işlem yapar, sonunda ürün çıkar. Faturalar okunur → ödemeler bulunur → makbuz ya da hatırlatma çıkar. Unix'te buna **pipe and filter** denir (`cat dosya | grep hata | sort` gibi). Toplu ve sıralı işlenirse **batch sequential** (faturalama sistemleri). **Artı (4):** dönüşümler yeniden kullanılır, sezgiseldir, yeni adım eklemek kolay, sıralı ya da paralel yapmak basit. **Eksi:** herkesin anlayacağı **ortak veri formatı** lazım (Unix'te düz metin, her adım ayrıştırıp yeniden yazar, bu yavaşlatır), ve **etkileşimli sistemlere (GUI) uygun değil** (fare tıklamaları bir "akış" değildir).

Önemli fark: nesne modeli **daha soyuttur**, çünkü işlemlerin **sırasını** göstermez. Boru hattı ise sırayı açıkça gösterir.

Bir de şu kural: **eşzamanlılığa erken karar verme!** Önce modüllere böl, paralel mi sıralı mı çalışacağına gerçekleştirmede karar ver, çünkü sıralı programlar test etmesi çok daha kolaydır.

📝 Deftere not

- Organizasyon seçilince alt sistemleri **modüllere** ayırma kararı. Katı ayrım yok, ama modüller alt sistemlerden **küçük**.
- ⭐⭐ **Alt sistem ve modül farkı:**
  - **Sub-system:** kendi başına sistem, çalışması diğer alt sistemlere **bağlı değil**. Modüllerden oluşur, tanımlı **arayüzleri** var.
  - **Module:** diğer modüllere hizmet sunan ve onların hizmetlerini kullanan bileşen. **Bağımsız sistem sayılmaz**, daha basit bileşenlerden oluşur.
- ⭐⭐ **2 ayrıştırma stratejisi:**
  - Object-oriented decomposition: **iletişim kuran nesneler**. Modül = **private state + işlemler**.
  - Function-oriented pipelining: girdiyi çıktıya **dönüştüren** fonksiyonel modüller. Modül = **fonksiyonel dönüşüm**.

  İkisi de **sıralı bileşen** ya da **süreç** olarak gerçekleştirilebilir.
- ⭐ **Eşzamanlılığa erken bağlanma.** Sıralı programlar tasarlamak, gerçekleştirmek, doğrulamak ve test etmek daha kolay. Önce modüllere ayır, sıralı ya da paralele **gerçekleştirmede** karar ver.

- ⭐ **11.3.1 OO model:** iyi tanımlı arayüzlü **loosely coupled** nesneler, birbirinin hizmetini çağırır.
- **Şekil 11.5 (fatura sistemi):** Customer, Payment, **Invoice** (issue, sendReminder, acceptPayment, sendReceipt), Receipt. UML: ad + öznitelikler + işlemler (alt kısım). **Kesikli ok** = başka nesnenin öznitelik ya da hizmetini kullanır.
- ⭐⭐ **OO avantajları: gevşek bağlı** (gerçekleştirme diğerlerini etkilemeden değişir) · **gerçek dünya varlıkları**, yapı kolay **anlaşılır** · **yeniden kullanılabilir** · OO dilleri doğrudan gerçekleştirir.
- ⭐⭐ **OO dezavantajları:** nesneler diğerlerinin **adına ve arayüzüne açıkça başvurur** → arayüz değişirse **tüm kullanıcılara** etkisi değerlendirilmeli · **karmaşık varlıkları** nesne olarak göstermek zor.
- ⭐⭐ **11.3.2 Function-oriented pipeline / data-flow model:** dönüşümler girdi işler, çıktı üretir, veri akarken **dönüşür**. Her adım = **transform**. Sıralı ya da paralel, **öğe öğe** ya da **batch**.
- ⭐ Ayrı süreçler olunca: pipe and filter stili (**Unix**: pipes = veri kanalları, komutlar = dönüşümler). **Filter** = girdiden işleyebileceğini süzer.
- ⭐ Sıralı + toplu = batch sequential model. **Faturalama sistemleri** gibi veri işleme sistemlerinde yaygın.
- **Şekil 11.6:** Invoices → read issued invoices → identify payments (+ Payments) → issue receipts → Receipts / find payments due → issue payment reminder → Reminders.
- ⭐ OO model pipeline'dan **daha soyut**: işlemlerin **sırasını** içermez.
- ⭐⭐ **Pipeline avantajları (4):** (1) dönüşümlerin **yeniden kullanımı**, (2) **sezgisel** (girdi-çıktı düşüncesi), (3) yeni dönüşüm ekleyerek **evrim kolay**, (4) eşzamanlı ya da sıralı gerçekleştirmesi **basit**.
- ⭐⭐ **Pipeline dezavantajları: ortak veri aktarım formatı** şart (Unix: **karakter dizisi**, her adım **parse / unparse** → ek yük, uyumsuz formatlar entegre edilemez) · **etkileşimli sistemler** zor (GUI = olaylar: fare tıklaması, menü seçimi, akışa çevrilemez).

Sayfa 256–260 — 11.4 Control styles · Centralised control (Şekil 11.7, 11.8) · Event-driven (Şekil 11.9, 11.10)

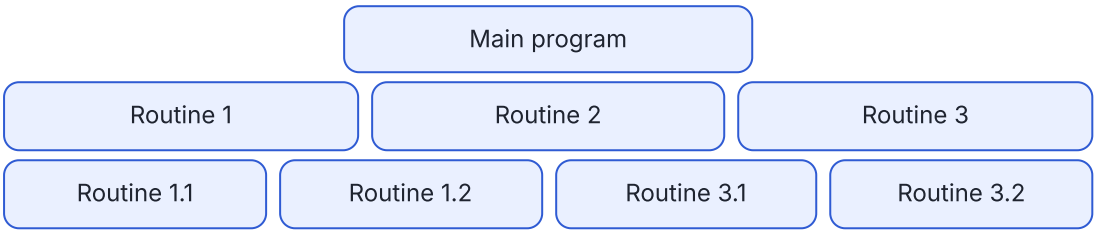
📖 Sayfanın özeti

KONTROL STİLLERİ

Sistemi yapılandırma modelleri sistemin alt sistemlere nasıl ayrıldığıyla ilgilidir. Bir sistem olarak çalışabilmeleri için alt sistemler, hizmetlerinin **doğru yere, doğru zamanda** ulaşması için kontrol edilmelidir. Yapısal modeller kontrol bilgisi **içermez (ve içermemelidir)**. Mimar, alt sistemleri kullanılan yapı modelini tamamlayan bir **kontrol modeline** göre düzenlemelidir. Mimari seviyedeki kontrol modelleri alt sistemler arası **kontrol akışıyla** ilgilenir. **2 genel kontrol stili** vardır: **1) Centralised control:** bir alt sistem kontrolün **genel sorumluluğunu** taşır, diğer alt sistemleri başlatır ve durdurur. Kontrolü başka bir alt sisteme devredebilir ama bu sorumluluğun kendisine geri dönmesini bekler. **2) Event-based control:** kontrol bilgisi bir alt sisteme gömülü olmak yerine her alt sistem **dışarıdan üretilen olaylara** yanıt verebilir. Bu olaylar diğer alt sistemlerden ya da sistemin ortamından gelebilir. Kontrol stilleri yapısal stillerle **birlikte** kullanılır. Anlatılan bütün yapısal stiller merkezi ya da olay tabanlı kontrolle gerçekleştirilebilir.

11.4.1 MERKEZİ KONTROL

Bir alt sistem **sistem denetleyicisi (controller)** olarak atanır ve diğer alt sistemlerin çalışmasını yönetmekten sorumludur. Kontrol edilen alt sistemlerin sıralı mı paralel mi çalıştığına göre **iki sınıf** vardır: **1) Call-return model:** bilinen **yukarıdan aşağı alt program (subroutine)** modelidir. Kontrol bir alt program hiyerarşisinin **tepesinden** başlar ve alt program çağrılarıyla ağacın alt seviyelerine geçer. Sadece **sıralı** sistemlere uygulanabilir. **2) Manager model: eşzamanlı (concurrent)** sistemlere uygulanabilir. Bir bileşen **sistem yöneticisi** olarak atanır ve diğer süreçlerin başlatılmasını, durdurulmasını ve koordinasyonunu kontrol eder. Süreç, diğer süreçlerle paralel çalışabilen bir alt sistem ya da modüldür. Bu modelin bir biçimi, bir yönetim rutininin bazı durum değişkenlerinin değerlerine göre belirli alt sistemleri çağırdığı **sıralı** sistemlerde de uygulanabilir. Bu genelde bir **case** ifadesiyle gerçekleştirilir.



Şekil 11.7: Call-return kontrol modeli

CALL-RETURN AYRINTILARI

Ana program Rutin 1, 2 ve 3'ü çağırabilir, Rutin 1 Rutin 1.1 ya da 1.2'yi, Rutin 3 Rutin 3.1 ya da 3.2'yi çağırabilir, vb. Bu, **program dinamiğinin** bir modelidir, **yapısal bir model değildir** (Rutin 1.1'in Rutin 1'in parçası olması gerekmez). Bu model C, Ada ve Pascal gibi programlama dillerine gömülüdür. Kontrol hiyerarşide üst seviyedeki bir rutinden alt seviyedekine geçer, sonra rutinin çağrıldığı noktaya **geri döner**. O an çalışan alt program kontrolden sorumludur, başka rutinleri çağırabilir ya da kontrolü üst rutine geri verebilir. Programın başka bir noktasına dönmek kötü programlama stilidir. Bu model modül seviyesinde fonksiyonları ya da nesneleri kontrol etmek için kullanılabilir. Programlama dillerindeki alt programlar doğal olarak fonksiyoneldir, ama birçok nesne yönelimli sistemde nesnelerdeki işlemler (metotlar) prosedür ya da fonksiyon olarak gerçekleştirilir (ör. bir Java nesnesi başka bir nesneden hizmet isteyince ilgili metodu çağırır). Bu modelin **katı ve sınırlı** yapısı hem bir **güç** hem bir **zayıflıktır**: kontrol akışlarını analiz etmek ve sistemin belirli girdilere nasıl yanıt vereceğini bulmak nispeten basit olduğu için güçtür. Normal çalışmanın **istisnalarını** ele almak zahmetli olduğu için zayıflıktır.

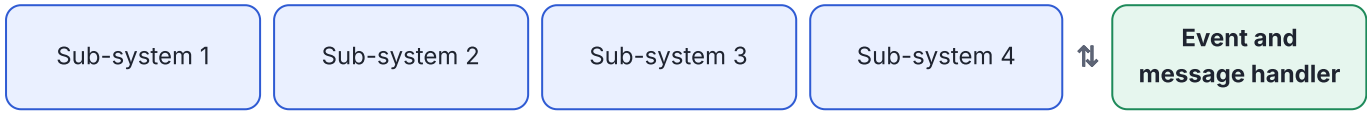
MANAGER MODEL (ŞEKİL 11.8)

Şekil 11.8 eşzamanlı bir sistem için **merkezi yönetim modelini** gösterir. Bu model çok sıkı zaman kısıtları olmayan **"yumuşak" gerçek zamanlı** sistemlerde sık kullanılır. Merkezi denetleyici, **sensörler ve aktüatörlerle** ilişkili süreçlerin çalışmasını yönetir (Bölüm 15'teki bina izleme sistemi bu modeli kullanır). Bileşenler: **system controller** merkezde, etrafında sensor processes, actuator processes, computation processes, user interface ve fault handler. Denetleyici süreci, süreçlerin ne zaman başlatılıp durdurulacağına **sistem durum değişkenlerine** göre karar verir. Diğer süreçlerin işlenecek bilgi üretip üretmediğini kontrol eder ya da işlenmek üzere onlara bilgi aktarır. Denetleyici genelde olayları ya da durum değişikliklerini yoklamak için sensörleri ve diğer süreçleri **sürekli döngüyle yoklar (polling)**. Bu yüzden bu modele bazen **event-loop** modeli denir.

11.4.2 OLAY GÜDÜMLÜ SİSTEMLER

Merkezi kontrol modellerinde kontrol kararları genelde bazı sistem **durum değişkenlerinin** değerleriyle belirlenir. Olay güdümlü kontrol modelleri ise **dışarıdan üretilen olaylarla** yönlendirilir. Bu bağlamda **olay** sadece ikili bir sinyal değildir, belirli değer aralığı alabilen bir sinyal ya da menüden

gelen bir komut girdisi olabilir. Olay ile basit bir girdi arasındaki fark, olayın **zamanlamasının** onu işleyen sürecin kontrolü **dışında** olmasıdır. Birçok tür olay güdümlü sistem vardır: kullanıcı arayüzü olaylarının düzenleme komutlarını gösterdiği **editörler**, bir koşulun doğru olmasının bir eylemi tetiklediği yapay zekâdaki **kural tabanlı üretim sistemleri** ve bir nesnenin özniteliğinin değişmesinin bazı eylemleri tetiklediği **aktif nesneler**. Bu bölümde **iki olay güdümlü model** anlatılıyor: **1) Broadcast models:** bir olay **bütün alt sistemlere yayınlanır**. O olayı işleyecek şekilde programlanmış herhangi bir alt sistem yanıt verebilir. **2) Interrupt-driven models:** sadece **gerçek zamanlı** sistemlerde kullanılır. Dış **kesmeler (interrupts)** bir kesme işleyicisi tarafından algılanır ve işlenmek üzere başka bir bileşene aktarılır. Broadcast modelleri ağdaki farklı bilgisayarlara **dağıtılmış** alt sistemleri entegre etmede etkilidir. Interrupt modelleri **katı zamanlama** gereksinimli gerçek zamanlı sistemlerde kullanılır.



Şekil 11.9: Seçici yayına (selective broadcasting) dayalı kontrol modeli

BROADCAST AYRINTILARI

Broadcast modelinde alt sistemler belirli olaylara **ilgi kaydeder (register)**. Bu olaylar gerçekleştiğinde kontrol, olayı işleyebilecek alt sisteme geçer. Bu modelin merkezi modelden farkı, kontrol politikasının olay ve mesaj işleyicisine **gömülü olmamasıdır**: alt sistemler hangi olaylara ihtiyaç duyduklarına karar verir, olay ve mesaj işleyici bu olayların onlara gönderilmesini sağlar. Bütün olaylar bütün alt sistemlere yayınlanabilir, ama bu çok fazla işlem yükü getirir. Daha çok, olay ve mesaj işleyici alt sistemlerin ve onların ilgilendiği olayların bir **kaydını (register)** tutar. Alt sistemler olay üretir (ör. bazı verilerin işlenmeye hazır olduğu). Olay işleyici olayları algılar, olay kaydına bakar ve olayı ilgilendiğini belirten alt sistemlere iletir. Kullanıcı arayüzü olaylarıyla yönlendirilen PC tabanlı basit sistemlerde fare, klavye vb. olaylarını dinleyip bunları daha özel komutlara çeviren açık **olay dinleyici (event-listener)** alt sistemleri vardır. Olay işleyici genelde **noktadan noktaya (point-to-point)** iletişimi de destekler: bir alt sistem açıkça başka bir alt sisteme mesaj gönderebilir. Bu modelin varyasyonları (ör. Field ortamı, HP Softbench) yazılım mühendisliği ortamlarında araç etkileşimlerini kontrol etmek için kullanılmıştır. Bölüm 12'deki **ORB (Object Request Broker)**'lar da dağıtık nesne iletişimi için bu kontrol modelini destekler.

BROADCAST: AVANTAJ VE DEZAVANTAJ

Avantajı: evrim nispeten basittir. Belirli olay sınıflarını işleyecek yeni bir alt sistem, olaylarını olay işleyiciye kaydettirerek entegre edilebilir. Herhangi bir alt sistem diğer bir alt sistemi onun **adını ya da yerini bilmeden** etkinleştirebilir. Alt sistemler dağıtık makinelerde gerçekleştirilebilir ve bu dağıtım diğer alt sistemlere **şeffaftır**. **Dezavantajı:** alt sistemler olayların **işlenip işlenmeyeceğini ya da ne zaman işleneceğini bilmez**. Bir alt sistem bir olay ürettiğinde o olaya hangi alt sistemlerin ilgi kaydettiğini bilmez. Farklı alt sistemlerin aynı olaylara kaydolması da oldukça mümkündür, bu da olayı işlemenin sonuçları kullanılabilir hale geldiğinde **çatışmalara** yol açabilir.

INTERRUPT-DRİVEN (ŞEKİL 11.10)

Dışarıdan üretilen olayların **çok hızlı** işlenmesini gerektiren gerçek zamanlı sistemler olay güdümlü olmalıdır. Ör: bir arabadaki emniyet sistemlerini kontrol eden gerçek zamanlı sistem olası bir çarpışmayı algılayıp sürücünün kafası direksiyona çarpmadan **hava yastığını** şişirmelidir. Bu hızlı yanıtı sağlamak için kesme güdümlü kontrol kullanılmalıdır. Şekil 11.10'da bilinen sayıda kesme türü vardır ve her tür için bir **işleyici (handler)** tanımlıdır. Her kesme türü, işleyicisinin adresinin saklandığı bellek konumuyla ilişkilidir (**interrupt vector**). Belirli türde bir kesme alındığında bir donanım anahtarı

kontrolün hemen işleyicisine geçmesini sağlar. Bu işleyici, kesmenin işaret ettiği olaya yanıt olarak başka süreçleri başlatabilir ya da durdurabilir. Bu model çoğunlukla bir olaya **anında yanıt** gereken gerçek zamanlı sistemlerde kullanılır. **Merkezi yönetim modeliyle birleştirilebilir**: merkezi yönetici sistemin normal çalışmasını yönetir, acil durumlar için kesme tabanlı kontrol kullanılır.



Şekil 11.10: Kesme güdümlü kontrol modeli

INTERRUPT: AVANTAJ VE DEZAVANTAJ

Avantajı: olaylara **çok hızlı yanıt** verilebilmesi. **Dezavantajları:** programlaması **karmaşık** ve **doğrulaması zordur**. Sistem testi sırasında kesme zamanlama kalıplarını tekrar oluşturmak imkânsız olabilir. Bu modelle geliştirilmiş sistemleri **değiştirmek zor** olabilir. Kesme sayısı **donanımla sınırlıysa**, bu sınıra ulaşıldığında başka türde olay işlenemez. Bu sınır bazen birkaç olay türünü tek bir kesmeye eşleyerek aşılabilir, işleyici hangi olayın olduğunu sonra çözer. Ama tek tek kesmelere **çok hızlı** yanıt gerekiyorsa kesme eşleme pratik olmayabilir.

👩‍🏫 Hoca anlatımı

Yapı modelleri "parçalar neler" der, **kontrol modelleri** ise "**kim kimi ne zaman çalıştırıyor**" der. İkisi birlikte kullanılır.

2 ana stil: Centralised = bir **patron** var, herkese ne yapacağını o söyler. **Event-based** = patron yok, herkes kendi ilgilendiği **olaya** tepki verir.

Centralised'ın 2 türü:

Call-return = şirket hiyerarşisi. Genel müdür bir müdüre iş verir, müdür bir şefe iş verir, şef işi bitirince müdüre **geri döner**, müdür genel müdüre döner. Programlamadaki fonksiyon çağrısı tam olarak budur. Sadece **sıralı** sistemlerde. **Güç:** akışı izlemek kolay. **Zayıflık:** istisnalar (beklenmedik durumlar) zahmetli.

Manager model = bir **orkestra şefi**. Aynı anda çalan (paralel) müzisyenleri yönetir, kimin ne zaman başlayıp duracağına karar verir. Sürekli etrafı yoklar ("sensörden yeni veri var mı?"), bu yüzden **event-loop** modeli de denir. "Yumuşak" gerçek zamanlı sistemlerde kullanılır (ör. bina izleme).

Event-based'ın 2 türü:

Broadcast = **okul anonsu**. Anons herkese gider, ilgilenen tepki verir. Daha akıllı versiyonu (Şekil 11.9): alt sistemler "beni şu olaylar ilgilendiriyor" diye **kaydolur**, olay işleyici olayı sadece onlara gönderir. Instagram'da birini takip etmek gibi: sadece takip ettiklerinin bildirimi sana gelir. **Artı:** yeni alt sistem eklemek kolay (sadece kaydol), kimin nerede olduğunu bilmeye gerek yok, dağıtık sistemlere uygun. **Eksi:** olayı gönderen, onu kimin ve ne zaman işleyeceğini bilmez, aynı olayı birden çok kişi işlerse çatışma olabilir.

Interrupt-driven = **yangın alarmı**. Ne yapıyorsan bırakıp hemen tepki verirsin. Gerçek zamanlı ve hayati sistemler için: **hava yastığı** örneğini unutma, sürücünün kafası direksiyona çarpmadan açılmalı. **Artı: çok hızlı**. **Eksi:** programlaması ve test etmesi zor (kesmelerin zamanlamasını testte tekrar oluşturamazsın), değiştirmek zor, kesme sayısı donanımla sınırlı.

Alıştırma 11.6 ve 11.7 bu konudan: maaş bordrosu (batch) → **call-return**. Farklı firmaların araçları birlikte çalışacak → **broadcast**. TV kumandası → **interrupt-driven**.

📝 Deftere not

- ⭐ Alt sistemler hizmetleri **doğru yere, doğru zamanda** vermek için kontrol edilmeli. Yapısal modeller **kontrol bilgisi içermez**. Mimar bir **kontrol modeli** ekler (alt sistemler arası **kontrol akışı**).
- ⭐⭐ **2 genel kontrol stili:**
  - Centralised control: bir alt sistem **genel kontrol sorumluluğunu** taşır, diğerlerini başlatır ve durdurur. Kontrolü devredebilir ama geri bekler.
  - Event-based control: her alt sistem **dışarıdan gelen olaylara** yanıt verir (diğer alt sistemlerden ya da ortamdan).

  Kontrol stilleri **yapısal stillerle birlikte** kullanılır.
- ⭐⭐ **Merkezi kontrolün 2 türü:**
  - Call-return model: **yukarıdan aşağı subroutine** modeli. Kontrol hiyerarşinin tepesinden başlar, alt programlara iner, çağrılan noktaya **geri döner**. Sadece **sıralı** sistemler. C, Ada, Pascal'a gömülü.
  - Manager model: **eşzamanlı** sistemler. Bir bileşen **sistem yöneticisi**: süreçleri başlatır, durdurur, koordine eder. Sıralı sistemde: durum değişkenine göre **case** ifadesi.
- ⭐ **Şekil 11.7:** Main program → Routine 1, 2, 3 → Routine 1.1, 1.2, 3.1, 3.2. **Program dinamiği** modeli, **yapısal model değil**.
- ⭐ Call-return'ün **katı yapısı**: **güç** = kontrol akışını analiz etmek kolay. **Zayıflık** = **istisnaları** ele almak zahmetli. Modül seviyesinde fonksiyon ve nesneler için de kullanılır (Java metot çağrısı).
- ⭐⭐ **Şekil 11.8, merkezi kontrol (gerçek zamanlı): system controller** + sensor processes, actuator processes, computation processes, user interface, fault handler. **"Soft" real-time** sistemlerde. Denetleyici **durum değişkenlerine** göre süreçleri başlatır ve durdurur, sensörleri sürekli **yoklar (polling)** → event-loop model. Ör: bina izleme sistemi.
- ⭐ **Merkezi:** karar = **durum değişkenleri**. **Olay güdümlü:** karar = **dış olaylar**.
- ⭐ **Event:** sadece ikili sinyal değil (değer aralığı, menü komutu). Olay ile girdi farkı: olayın **zamanlaması** onu işleyen sürecin **kontrolü dışında**.
- Olay güdümlü sistem türleri: **editörler** (UI olayları), **kural tabanlı üretim sistemleri** (yapay zekâ), **aktif nesneler**.
- ⭐⭐ **2 olay güdümlü model:**
  - Broadcast models: olay **tüm alt sistemlere** yayınlanır, işleyebilen yanıt verir. **Dağıtık** alt sistemleri entegre etmede etkili.
  - Interrupt-driven models: sadece **real-time**. Dış **kesmeleri** kesme işleyicisi algılar, başka bileşene aktarır. **Katı zamanlama** gereksinimi.
- ⭐ **Şekil 11.9, selective broadcasting:** alt sistemler olaylara **ilgi kaydeder**. **Event and message handler** bir **kayıt (register)** tutar, olayı sadece ilgilenenlere iletir. Kontrol politikası işleyicide **gömülü değil**. **Point-to-point** mesajı da destekler. Ör: event-listener (fare, klavye), Field, HP Softbench, **ORB**'lar.
- ⭐⭐ **Broadcast avantajı: evrim basit** (yeni alt sistem olaylarını kaydettirir) · başka alt sistemi **adını ya da yerini bilmeden** etkinleştirir · dağıtım **şeffaf**.
- ⭐⭐ **Broadcast dezavantajı:** alt sistem olayın **işlenip işlenmeyeceğini ve ne zaman işleneceğini bilmez** · kimin kayıtlı olduğunu bilmez · aynı olaya birden çok kayıt → **çatışma**.

- ⭐⭐ **Interrupt-driven (Şekil 11.10):** çok hızlı yanıt gereken real-time (ör. **hava yastığı**). Interrupts → **interrupt vector** (işleyici adresleri) → handlers → processes. Donanım anahtarı kontrolü **hemen** işleyiciye geçirir. **Merkezi modelle birleşebilir** (normal çalışma merkezi, acil durumlar kesmeyle).
- ⭐⭐ **Interrupt avantajı: çok hızlı yanıt**. **Dezavantajları:** programlaması **karmaşık**, **doğrulaması zor** (zamanlama kalıpları testte tekrarlanamaz), **değiştirmesi zor**, kesme sayısı **donanımla sınırlı** (birden çok olayı tek kesmeye eşleme, ama çok hızlı yanıtta pratik değil).

Sayfa 260–265 — 11.5 Reference architectures · OSI (Şekil 11.11) · ECMA CASE modeli (Şekil 11.12) · Key Points · Exercises

📖 Sayfanın özeti

ALANA ÖZGÜ MİMARİLER

Yukarıdaki mimari modeller **genel** modellerdir, birçok uygulama sınıfına uygulanabilir. Bunlara ek olarak belirli bir uygulama alanına **özgü** mimari modeller de kullanılabilir. Bu sistemlerin örnekleri ayrıntıda farklı olsa da ortak mimari yapı yeni sistemler geliştirirken yeniden kullanılabilir. Bunlara **alana özgü mimariler (domain-specific architectures)** denir. **İki tür** vardır: **1) Generic models:** bir dizi gerçek sistemden çıkarılan **soyutlamalardır**. Bu sistemlerin temel özelliklerini kapsarlar. Ör: gerçek zamanlı sistemlerde veri toplama ya da izleme sistemleri için genel mimari modeller olabilir (Bölüm 13). **2) Reference models: daha soyuttur** ve daha geniş bir sistem sınıfını tanımlar. Tasarımcılara o sistem sınıfının genel yapısı hakkında bilgi vermenin bir yoludur. Genelde uygulama alanının **incelenmesinden** türetilirler. Sistemlerin içerebileceği bütün özellikleri içeren **idealleştirilmiş** bir mimariyi temsil ederler.

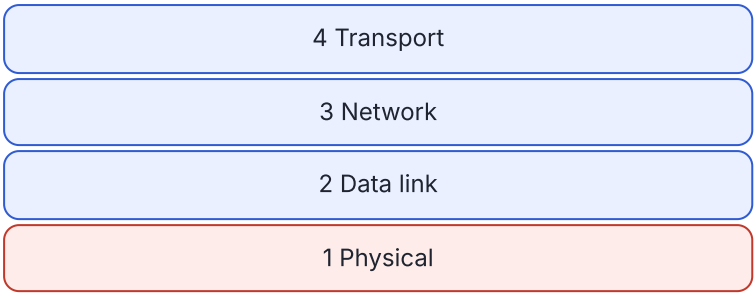
FARK VE KULLANIM

Bu iki model türü arasında elbette kesin bir ayrım yoktur, genel modeller de referans model işlevi görebilir. Yazar bunları ayırıyor, çünkü **genel modeller tasarımda doğrudan yeniden kullanılabilir**. **Referans modeller** ise normalde alan kavramlarını **iletmek** ve olası mimarileri **karşılaştırmak ya da değerlendirmek** için kullanılır. Referans mimariler normalde gerçekleştirmeye giden bir yol sayılmaz. Asıl işlevleri alana özgü mimarileri **tartışmak** ve bir alandaki farklı sistemleri **karşılaştırmaktır**. Referans model karşılaştırma için bir **sözlük (vocabulary)** sağlar ve sistemlerin değerlendirilebileceği bir **temel** görevi görür.

OSI MODELİ (ŞEKİL 11.11)

OSI, açık sistemler arası bağlantı (open systems interconnection) için **7 katmanlı** bir modeldir. Katmanlar (alttan üste): **1 Physical, 2 Data link, 3 Network, 4 Transport, 5 Session, 6 Presentation, 7 Application**. İki uç sistem 7 katmanın hepsine sahiptir, aradaki ağ düğümü sadece alt üç katmanı (physical, data link, network) kullanır ve hepsi iletişim ortamına bağlıdır. Katmanların tam işlevleri burada önemli değil. Özünde **alt katmanlar fiziksel bağlantıyla**, **orta katmanlar veri aktarımıyla**, **üst katmanlar** ise standartlaştırılmış belgeler gibi anlamlı uygulama bilgisinin aktarımıyla ilgilenir.





Şekil 11.11: OSI referans modeli (alt 3 katman = ağ düğümü de kullanır)

OSI: AMAÇ VE GERÇEK

OSI tasarımcılarının çok pratik bir hedefi vardı: uyumlu sistemlerin birbiriyle iletişim kurabilmesi için bir **gerçekleştirme standardı** tanımlamak. Her katman sadece **altındaki katmana** bağlı olmalıydı. Teknoloji geliştikçe bir katman, diğer katmanları kullanan sistemleri etkilemeden **şeffaf şekilde yeniden gerçekleştirilebilmeliydi**. Pratikte ise katmanlı yaklaşımın **performans sorunları** bu hedefi zayıflattı. Ağlar arasındaki büyük farklar yüzünden basit bağlantı imkânsız olabilir. Her katmanın fonksiyonel özellikleri iyi tanımlanmış olsa da **fonksiyonel olmayan özellikleri tanımlanmamıştır**. Geliştiriciler kendi üst seviye olanaklarını gerçekleştirip modeldeki **katmanları atlamak** ya da sistem performansını artırmak için standart dışı özellikler tasarlamak zorunda kalır. Sonuç olarak modeldeki bir katmanın şeffaf şekilde değiştirilmesi pratikte **neredeyse hiç mümkün değildir**. Ama bu, modelin **faydasını ortadan kaldırmaz**: sistemler arası iletişimin soyut yapılandırılması ve sistematik gerçekleştirilmesi için bir **temel** sağlar.

ECMA CASE REFERANS MODELİ (ŞEKİL 11.12)

Önerilen başka bir referans model **CASE ortamları** içindir (**ECMA**, Brown ve arkadaşları). Bir CASE ortamının sağlaması gereken **beş hizmet kümesini** belirler. Ayrıca bu hizmetleri kullanan tek tek CASE araçları için **"tak-çalıştır" (plug-in)** olanakları sağlamalıdır (şekilde araç yuvaları, tool slots). **5 hizmet seviyesi: 1) Data repository services:** veri öğelerinin ve ilişkilerinin **saklanması ve yönetimi**. **2) Data integration services:** grupları yönetmek ya da aralarında ilişki kurmak. Bunlar ve veri deposu hizmetleri ortamda **veri entegrasyonunun** temelidir. **3) Task management services: süreç modellerinin** tanımlanması ve uygulanması. **Süreç entegrasyonunu** destekler. **4) Message services:** araç-araç, ortam-araç ve ortam-ortam **iletişimi**. **Kontrol entegrasyonunu** destekler. **5) User interface services: kullanıcı arayüzü** geliştirme olanakları. **Sunum entegrasyonunu** destekler.

ECMA MODELİNİN KULLANIMI

Bu referans model herhangi bir CASE ortamına **nelerin dahil edilebileceğini** söyler, ama bir referans mimarinin her özelliğinin gerçek mimari tasarımlarda bulunmayacağını vurgulamak önemlidir. Bu, bir sistem tasarımı hakkında "veri deposu hizmetleri nasıl sağlanıyor?" ya da "sistem görev yönetimi sağlıyor mu?" gibi **sorular sorabileceğimiz** anlamına gelir. Bu referans mimarinin asıl değeri yine entegre CASE araç ve ortamlarını **sınıflandırmak ve karşılaştırmak** için bir araç olmasıdır. Ayrıca eğitimde bu ortamların temel özelliklerini vurgulamak ve onları genel bir şekilde tartışmak için kullanılabilir.

KEY POİNTS (BÖLÜM ÖZETİ)

Yazılım mimarisi sistemi yapılandırmanın temel çerçevesidir, performans, güvenlik ve availability gibi özellikler kullanılan mimariden etkilenir. Mimari tasarım kararları uygulamanın türü, sistemin dağıtımı, kullanılacak mimari stiller ve mimarinin nasıl belgelenip değerlendirileceği hakkındaki kararları içerir. Mimari tasarım sürecinde yapısal model, kontrol modeli ve ayrıştırma modeli gibi farklı modeller geliştirilebilir. Sistemin organizasyon modelleri repository, client-server ve soyut makine modellerini içerir: repository modelleri veriyi ortak bir depo üzerinden paylaşır, client-server modelleri genelde

veriyi dağıtır, soyut makine modelleri katmanlıdır ve her katman temel katmanının olanaklarıyla gerçekleştirilir. Ayrıştırma stilleri nesne yönelimli ve fonksiyon odaklı ayrıştırmayı içerir: boru hattı modelleri fonksiyoneldir, nesne modelleri kendi durumlarını ve işlemlerini tutan gevşek bağlı varlıklara dayanır. Kontrol stilleri merkezi kontrol ve olay tabanlı kontrolü içerir: merkezi modellerde kontrol kararları sistem durumuna göre, olay modellerinde dış olaylar sistemi kontrol eder. Referans mimariler alana özgü mimarileri tartışmak ve mimari tasarımları değerlendirip karşılaştırmak için bir araç olarak kullanılabilir.

FURTHER READİNG

Bass ve arkadaşlarının *Software Architecture in Practice*, Bosch'un *Design and Use of Software Architectures* (ürün hattı mimarileri) ve Shaw ile Garlan'ın ilk yazılım mimarisi kitabı *Software Architecture: Perspectives on an Emerging Discipline*. *Sınav için önemli değil.*

ALIŞTIRMALAR (KONULARI)

#	Konu
11.1	Sistem mimarisini tanımlamalar yazılmadan **önce** tasarlamak neden gerekebilir?
11.2	Availability ve security en önemli gereksinimler olduğunda mimari tasarımda ne tür **çatışmalar** çıkabilir?
11.3	Bu bölümdeki **yapısal modellerin** avantaj ve dezavantajlarını gösteren bir tablo hazırlayın.
11.4	Şu sistemler için uygun yapısal model önerin: tren istasyonunda otomatik **bilet satış** sistemi, birçok katılımcının video, ses ve veriyi gördüğü **video konferans** sistemi, koridor gibi açık alanları temizleyen, duvar ve engelleri algılayan **robot süpürge**.
11.5	Yukarıdaki sistemler için seçtiğiniz modele dayalı bir mimari tasarlayın.
11.6	Gerçek zamanlı sistemler genelde olay güdümlüdür. Hangi durumda gerçek zamanlı bir sistem için **call-return** kontrol modeli önerirsiniz?
11.7	Şu sistemler için uygun **kontrol modeli** önerin: çalışma saatleri ve ücretlerle **maaş bordrosu** basan toplu işlem sistemi, farklı firmaların ürettiği ama birlikte çalışması gereken **yazılım araçları**, uzaktan kumandaya yanıt veren **TV denetleyicisi**.
11.8	Tek makine ve dağıtık sürümler için **data-flow** ve **object** modellerinin dağıtılabilirlik açısından artı ve eksileri.
11.9	İki entegre CASE araç takımını karşılaştırmak için bir **CASE referans modelini** nasıl kullanırsınız?
11.10	Müşteriyle bağımsız çalışıp mimari tasarlayan ayrı bir "**yazılım mimarı**" mesleği olmalı mı? Zorlukları neler olur?

👨‍🏫 Hoca anlatımı

Generic ve reference model farkı: Generic = gerçek sistemlerden çıkarılmış **hazır kalıp**, doğrudan kullanırsın (bir ev planı şablonu gibi). **Reference** = daha soyut, **ideal** bir model. Doğrudan inşa etmek için değil, **karşılaştırmak, değerlendirmek ve konuşmak** için kullanılır (bir "ideal ev nasıl olmalı" rehberi gibi: "bu evde mutfak var mı, banyo var mı?" diye kontrol edersin).

OSI en ünlü referans model. 7 katmanı aşağıdan yukarı ezberlemek için klasik hatırlatıcı: "**P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way" → Physical, Data link, Network, Transport, Session,

Presentation, Application. Alt katmanlar = fiziksel bağlantı, orta = veri aktarımı, üst = uygulama bilgisi.

OSI'nin dersi: **teoride** her katman bağımsız değiştirilebilir. **Pratikte** performans sorunları yüzünden geliştiriciler katmanları atlar, bu yüzden şeffaf değiştirme neredeyse hiç olmaz. Ama model yine de çok faydalı (düşünme ve konuşma çerçevesi). Bu, layered modelin dezavantajlarının gerçek hayattaki örneği.

ECMA CASE modeli: bir CASE ortamının 5 hizmeti. Her biri bir entegrasyon türünü destekler: **Data repository + data integration → veri entegrasyonu**, **Task management → süreç entegrasyonu**, **Message → kontrol entegrasyonu**, **User interface → sunum entegrasyonu**. Alıştırma 11.9 bunu soruyor: iki CASE ürününü karşılaştırmak için bu 5 hizmet üzerinden "bu ürün bunu nasıl sağlıyor?" diye tek tek bakarsın.

Alıştırma 11.4 kısa cevaplar: bilet makinesi → **client-server** (makineler istemci, merkezi sunucu) ya da basit sistemse **repository**. Video konferans → **client-server** (dağıtık, birden çok katılımcı). Robot süpürge → **layered** ya da olay güdümlü (sensör, kontrol, motor katmanları).

📝 Deftere not

- ⭐ Domain-specific architectures: belirli uygulama alanına özgü, ortak yapı yeniden kullanılır.
- ⭐⭐ **2 tür:**
  - Generic models: gerçek sistemlerden **soyutlama**, temel özellikleri kapsar. Ör: real-time veri toplama ve izleme sistemleri (Bölüm 13). **Tasarımda doğrudan yeniden kullanılır.**
  - Reference models: **daha soyut**, daha geniş sistem sınıfı. Alan **incelemesinden** türer, **idealleştirilmiş** mimari. **İletişim**, **karşılaştırma ve değerlendirme** için.
- ⭐ Referans mimariler gerçekleştirme yolu **değil**. İşlevi: alana özgü mimarileri **tartışmak**, sistemleri **karşılaştırmak**. Karşılaştırma için **sözlük (vocabulary)** + değerlendirme **temeli**.
- ⭐⭐ **OSI (Şekil 11.11):** açık sistemler arası bağlantı için **7 katman**: 1 Physical · 2 Data link · 3 Network · 4 Transport · 5 Session · 6 Presentation · 7 Application. Ara ağ düğümü sadece alt 3 katman. **Alt** = fiziksel bağlantı, **orta** = veri aktarımı, **üst** = anlamlı uygulama bilgisi.
- ⭐ OSI amacı: **gerçekleştirme standardı**, her katman sadece **altındakine** bağlı, katman **şeffaf değiştirilebilir**.
- ⭐⭐ OSI gerçeği: katmanlı yaklaşımın **performans sorunları** + ağ farkları + katmanların **NFR'leri tanımsız** → geliştiriciler **katmanları atlar**, standart dışı özellik ekler → şeffaf değiştirme **neredeyse hiç mümkün değil**. Ama model hâlâ **faydalı** (soyut yapı ve sistematik iletişim temeli).
- ⭐⭐ **ECMA CASE referans modeli (Şekil 11.12):** 5 hizmet + araçlar için **plug-in** (tool slots):
  - **Data repository services:** veri öğeleri ve ilişkilerin saklanması ve yönetimi.
  - **Data integration services:** grupları yönetme, ilişki kurma. (1 + 2 = **veri entegrasyonu**)
  - **Task management services:** süreç modellerinin tanımı ve uygulanması → **süreç entegrasyonu**.
  - **Message services:** araç-araç, ortam-araç, ortam-ortam iletişimi → **kontrol entegrasyonu**.
  - **User interface services:** UI geliştirme → **sunum entegrasyonu**.
- Referansın her özelliği gerçek tasarımda olmaz. "Veri deposu nasıl sağlanıyor?", "görev yönetimi var mı?" gibi **sorular sorulabilir**. Asıl değer: CASE ortamlarını **sınıflandırma ve karşılaştırma** +

eğitim.

- ⭐⭐ **KEY POINTS:** mimari = sistemi yapılandırmanın **temel çerçevesi**, performans, güvenlik, availability'yi etkiler · kararlar: uygulama türü, dağıtım, stiller, belgeleme ve değerlendirme · modeller: **yapısal, kontrol, ayrıştırma** · organizasyon: **repository** (ortak depo), **client-server** (genelde dağıtık veri), **abstract machine** (katmanlı) · ayrıştırma: **OO** (gevşek bağlı, kendi durumu ve işlemleri) ve **function-oriented / pipeline** · kontrol: **centralised** (sistem durumu) ve **event-based** (dış olaylar) · referans mimariler: tartışma, değerlendirme, karşılaştırma.
- ⭐ Sınavda en olası: **11.3** (artı ve eksi tablosu), **11.4** (yapısal model seçimi), **11.7** (kontrol modeli seçimi), **11.2** (çatışma), **11.1**.
- ⭐ **11.7 kısa cevaplar:** maaş bordrosu (batch) → **call-return** · farklı firmaların araçları → **broadcast** · TV kumandası → **interrupt-driven**.

BÖLÜM 11 SONU: TEKRAR KISMI

Terim sözlüğü · ⭐ Mutlaka bil · Karşılaştırma tablosu · Olası sınav soruları · Mini test

A Terim sözlüğü

Bölüm 11'deki tüm terimler

İngilizce	Türkçe	Anlamı	Sayfa
software architecture	yazılım mimarisi	Sistemi yapılandırmanın temel çerçevesi	242
architectural design	mimari tasarım	Alt sistemleri ve kontrol / iletişim çerçevesini belirleme	242
sub-system	alt sistem	Kendi başına çalışabilen sistem parçası	242, 252
stakeholder communication	paydaş iletişimi	Mimarinin 1. avantajı	242
system analysis	sistem analizi	Mimarinin 2. avantajı	242
large-scale reuse	büyük ölçekli yeniden kullanım	Mimarinin 3. avantajı	242
product-line architecture	ürün hattı mimarisi	İlişkili sistemlerde aynı mimari	242, 245
complexity management	karmaşıklık yönetimi	Ayrıntıyı gizleyip ana soyutlamalara odaklanma	242
large-grain / fine-grain components	büyük taneli / ince taneli bileşenler	Büyük ve az parça / küçük ve çok parça	242–243
redundant components	yedekli bileşenler	Availability için	243
self-contained	kendi kendine yeten	Bağımsız bileşen	243
block diagram	blok diyagramı	Kutu = alt sistem, ok = veri / kontrol	243–244
box-and-line diagram	kutu-çizgi diyagramı	Basit mimari gösterim	244
packing robot	paketleme robotu	Şekil 11.1 örneği	243–244
localised change	yerel değişiklik	Değişikliğin tek alt sistemde kalması	244
architectural style	mimari stil	Sistem organizasyonu kalıbı (Garlan & Shaw)	246
composite architecture	bileşik mimari	Birden çok stilin birleşimi	246

static structural model	statik yapısal model	Ayrı geliştirilecek alt sistemler	246
dynamic process model	dinamik süreç modeli	Çalışma zamanındaki süreçler	246
interface model	arayüz modeli	Alt sistemlerin sunduğu hizmetler	246
relationship model	ilişki modeli	Alt sistemler arası veri akışı vb.	246
distribution model	dağıtım modeli	Alt sistemlerin bilgisayarlara dağılımı	246
ADL (architectural description language)	mimari tanımlama dili	Components + connectors, uzman dili	246–247
components / connectors	bileşenler / bağlayıcılar	ADL'lerin temel öğeleri	247
system organisation	sistem organizasyonu	Sistemi yapılandırmanın temel stratejisi	247
repository model	depo modeli	Paylaşılan merkezi veritabanı	247
project repository	proje deposu	CASE araç takımının merkezi	248
schema / data model	şema / veri modeli	Deponun ortak veri yapısı	248
blackboard model	kara tahta modeli	Veri hazır olunca alt sistemleri tetikleyen aktif depo	249
client-server model	istemci-sunucu modeli	Hizmetler, sunucular, istemciler, ağ	249
server / client	sunucu / istemci	Hizmet sunan / hizmet isteyen	249
remote procedure call	uzak prosedür çağrısı	İstemcinin sunucuya erişim yolu	249
request-reply protocol	istek-cevap protokolü	Ör. http	249
XML	XML	Şema dönüşümü kolay ama verimsiz veri gösterimi	250
layered model / abstract machine model	katmanlı model / soyut makine modeli	Her katman bir hizmet kümesi	250
APSE	Ada programlama destek ortamı	Buxton'ın 3 katmanlı modeli	251

portable	taşınabilir	Farklı platforma kolay geçen	251
punch through	delip geçmek	Katmanları atlayarak alt katmana erişmek	252
modular decomposition	modüler ayrıştırma	Alt sistemleri modüllere bölme	252
module	modül	Bağımsız sistem sayılmayan, hizmet sunan bileşen	252
object-oriented decomposition	nesne yönelimli ayrıştırma	İletişim kuran nesnelere bölme	252–253
loosely coupled	gevşek bağlı	Birbirine az bağımlı	253
concurrency	eşzamanlılık	Paralel çalışma (erken karar verme)	253
attributes / operations	öznitelikler / işlemler	Nesne sınıfının bilgisi ve davranışları	253
invoice / receipt / reminder	fatura / makbuz / hatırlatma	Örnek fatura sistemi	253–255
function-oriented pipelining	fonksiyon odaklı boru hattı	Veriyi dönüştüren adımlar dizisi	254
data-flow model	veri akış modeli	Pipeline'ın diğer adı	254
transform	dönüşüm	Bir işleme adımı	254
pipe and filter	boru ve süzgeç	Unix tarzı pipeline	254
batch sequential	toplu sıralı	Sıralı + toplu veri işleme (faturalama)	254
parse / unparse	ayrıştırma / geri biçimleme	Pipeline'da veri formatı yükü	255
control styles	kontrol stilleri	Alt sistemler arası kontrol akışı	256
centralised control	merkezi kontrol	Bir alt sistem diğerlerini yönetir	256
event-based control	olay tabanlı kontrol	Alt sistemler dış olaylara tepki verir	256
call-return model	çağır-dön modeli	Yukarıdan aşağı subroutine, sıralı	256–257
subroutine	alt program	Çağrılan rutin	256

manager model	yönetici modeli	Eşzamanlı süreçleri yöneten merkezi bileşen	256
state variables	durum değişkenleri	Merkezi kontrol kararlarının dayanağı	256–257
sensors / actuators	sensörler / eyleyiciler	Gerçek zamanlı sistem bileşenleri	257–258
soft real-time	yumuşak gerçek zamanlı	Çok sıkı zaman kısıtı olmayan	257
polling / event-loop	yoklama / olay döngüsü	Denetleyicinin sürekli kontrolü	257
event	olay	Zamanlaması işleyenin kontrolü dışındaki sinyal	258
broadcast model	yayın modeli	Olay alt sistemlere yayınlanır	258
event and message handler	olay ve mesaj işleyici	Olayları ilgilenenlere ileten bileşen	259
register (interest)	ilgi kaydı	Alt sistemin hangi olayları istediği	259
point-to-point	noktadan noktaya	Doğrudan mesaj	259
ORB (Object Request Broker)	nesne istek aracısı	Dağıtık nesne iletişimi	259
interrupt-driven model	kesme güdümlü model	Kesmelerle hızlı yanıt (real-time)	258, 260
interrupt / interrupt handler / interrupt vector	kesme / işleyici / kesme vektörü	Kesme ve işleyici adresleri	260
reference architecture	referans mimari	Karşılaştırma ve değerlendirme için ideal model	260–261
domain-specific architecture	alana özgü mimari	Belirli alana özgü yapı	261
generic model	genel model	Gerçek sistemlerden soyutlama, doğrudan kullanılır	261
OSI reference model	OSI referans modeli	7 katmanlı ağ modeli	261
ECMA reference architecture	ECMA referans mimarisi	CASE ortamları için 5 hizmet	262
tool slots / plug-in	araç yuvaları / eklenti	CASE araçlarının takıldığı yerler	262

B ⭐ Mutlaka bil

deftere elle yaz

Bölüm 11'den sınavda en çok çıkabilecekler

1. **Mimari tasarım tanımı** (alt sistemler + kontrol ve iletişim çerçevesi), tasarımın ilk aşaması (s.242)
2. **3 avantaj (Bass):** stakeholder communication, system analysis, large-scale reuse (s.242)
3. **NFR'ye göre mimari:** performance (large-grain, az iletişim), security (katman, değerli en içte), safety (tek alt sistem), availability (redundant), maintainability (fine-grain, paylaşılan veri yok) + **çatışma** (s.242–243)
4. **Blok diyagramı** ve Bass'ın eleştirisi / yazarın savunması (s.243–244)
5. **8 karar sorusu** (s.245)
6. **Mimari stil** tanımı, 3 karar grubu: organizasyon, ayrıştırma, kontrol (s.246)
7. **5 mimari model:** static, dynamic, interface, relationship, distribution · **ADL** (s.246–247)
8. **Repository**: tanım, örnek (CASE), **4 artı / 4 eksi** · blackboard (s.247–249)
9. **Client-server**: 3 bileşen, request-reply, film kütüphanesi örneği, artı / eksi (s.249–250)
10. **Layered**: soyut makine, Şekil 11.4, artılar (artımlı, değiştirilebilir, taşınabilir) / eksiler (punch through, performans) (s.250–252)
11. **Alt sistem ve modül farkı** (s.252)
12. **OO decomposition** artı / eksi · **pipeline** (pipe and filter, batch sequential) 4 artı / 2 eksi (s.253–255)
13. **Centralised** (call-return, manager) ve **event-based** (broadcast, interrupt-driven) + her birinin artı / eksisi (s.256–260)
14. **Generic ve reference model** farkı · **OSI 7 katman** + pratik sorunları · **ECMA 5 hizmeti** (s.260–263)

C Karşılaştırma tablosu (Alıştırma 11.3)

deftere geçir

Model	Avantajlar	Dezavantajlar
Repository	Büyük veriyi paylaşmak verimli · veri üreten, kullanımla ilgilenmez · yedekleme, güvenlik, kurtarma merkezi · uyumlu yeni araç eklemek kolay	Ortak veri modeli = uzlaşma, performans düşebilir · evrim zor · herkese aynı politika · dağıtmak zor
Client-server	Dağıtık, ağdaki işlemcileri kullanır · yeni sunucu eklemek ve sunucuyu şeffaf yükseltmek kolay	Ortak veri modeli yok · değişiklik istemcileri de etkileyebilir · XML dönüşümü verimsiz
Layered	Artımlı geliştirme · arayüz aynıysa katman değiştirilebilir · değişiklik sadece komşuyu etkiler · taşınabilir, çok platform	Yapılandırmak zor (punch through) · çoklu yorumlama yüzünden performans
Object-oriented	Gevşek bağlı, nesne değişince diğerleri etkilenmez · anlaşılır (gerçek dünya) · yeniden kullanılabilir · OO dilleri	Nesneler birbirinin adını ve arayüzünü bilmeli · karmaşık varlıkları göstermek zor
Pipeline	Dönüşümler yeniden kullanılır · sezgisel · yeni dönüşüm eklemek kolay · sıralı ya da paralel kolay	Ortak veri formatı şart (parse / unparse yükü) · etkileşimli (GUI) sistemlere uygun değil
Call-return	Kontrol akışını analiz etmek kolay	İstisnaları ele almak zahmetli · sadece sıralı
Broadcast	Evrim kolay (kayıt yeter) · ad ve yer bilmeden etkinleştirme · dağıtım şeffaf	Olayın işlenip işlenmeyeceği ve ne zaman işleneceği bilinmez · çatışma
Interrupt-driven	Çok hızlı yanıt	Karmaşık programlama · doğrulaması ve değiştirmesi zor · kesme sayısı donanımla sınırlı

D Olası sınav soruları

İngilizce cevap kalıbı + Türkçesi

Q1. What is architectural design? What are the advantages of explicitly designing a software architecture?

Mimari tasarım nedir, avantajları? (s.242)

> *Architectural design is the initial design process of identifying the sub-systems of a system and establishing a framework for sub-system control and communication. Its output is a description of the software architecture. Advantages: 1) stakeholder communication: it is a high-level presentation for discussion, 2) system analysis: early decisions affect performance, reliability and maintainability, 3) large-scale reuse: similar systems can use the same architecture.*

Mimari tasarım, sistemin alt sistemlerini belirleyip alt sistemlerin kontrolü ve iletişimi için bir çerçeve kuran ilk tasarım sürecidir. Çıktısı yazılım mimarisinin tanımıdır. Avantajları: 1) paydaş iletişimi: tartışma için üst seviye sunum, 2) sistem analizi: erken kararlar performans, güvenilirlik ve bakımı etkiler, 3) büyük ölçekli yeniden kullanım: benzer sistemler aynı mimariyi kullanabilir.

Q2. How do non-functional requirements influence the architecture?
NFR'ler mimariyi nasıl etkiler? (s.242–243)

> *Performance: localise critical operations in a small number of large-grain sub-systems with little communication. Security: use a layered structure with critical assets in the innermost layers. Safety: put safety-related operations in a single sub-system or a small number of sub-systems. Availability: include redundant components so they can be replaced without stopping the system. Maintainability: use fine-grain, self-contained components and avoid shared data structures. These can conflict, for example large-grain components help performance but fine-grain components help maintainability.*

Performans: kritik işlemleri az sayıda büyük taneli, az iletişimli alt sistemde topla. Güvenlik: kritik varlıklar en iç katmanda olacak şekilde katmanlı yapı. Emniyet: emniyet işlemlerini tek ya da az sayıda alt sistemde topla. Erişilebilirlik: sistemi durdurmadan değiştirilebilen yedekli bileşenler. Bakım: ince taneli, bağımsız bileşenler, paylaşılan veri yapısı yok. Bunlar çatışabilir: büyük taneli bileşen performansa, ince taneli bakıma yardım eder.

Q3. Explain why design conflicts might arise when availability and security are the most important requirements. (Ex. 11.2)
Availability ve security çatışması

> *Availability needs redundant components and the ability to replace and update components without stopping the system, so there are more components and more access paths. Security needs a layered structure with critical assets protected in the innermost layers and strong validation, which limits access and adds checks. More redundancy and easy replacement can create more vulnerabilities, while strict security checks can slow down or block recovery. A compromise must be found.*

Availability, yedekli bileşenler ve sistemi durdurmadan bileşen değiştirme ister, yani daha çok bileşen ve erişim yolu demektir. Security ise kritik varlıkların en içte korunduğu katmanlı yapı ve sıkı doğrulama ister, erişimi kısıtlar ve kontrol ekler. Fazla yedeklilik ve kolay değiştirme daha çok zafiyet yaratabilir, sıkı güvenlik kontrolleri ise kurtarmayı yavaşlatabilir. Bir uzlaşma bulunmalıdır.

Q4. What fundamental questions must be answered during architectural design?
Mimari tasarımda hangi sorular cevaplanır? (s.245)

> *Is there a generic application architecture to use as a template? How will the system be distributed across processors? Which architectural styles are appropriate? What is the fundamental approach to structure the system? How will the structural units be decomposed into modules? What strategy will control the units? How will the design be evaluated? How should it be documented?*

Şablon olabilecek genel bir uygulama mimarisi var mı? Sistem işlemcilere nasıl dağıtılacak? Hangi mimari stiller uygun? Sistemi yapılandırmanın temel yaklaşımı ne? Yapısal birimler modüllere nasıl ayrılacak? Birimler hangi stratejiyle kontrol edilecek? Tasarım nasıl değerlendirilecek? Nasıl belgelenecek?

Q5. What architectural models may be developed?
Hangi mimari modeller geliştirilebilir? (s.246)

> *1) A static structural model showing the sub-systems or components. 2) A dynamic process model showing how the system is organised into processes at run-time. 3) An interface model defining the services of each sub-system. 4) Relationship models showing relationships such as data flow. 5) A distribution model showing how sub-systems are distributed across computers.*

1) Alt sistemleri ya da bileşenleri gösteren statik yapısal model. 2) Çalışma zamanındaki süreçleri gösteren dinamik süreç modeli. 3) Her alt sistemin hizmetlerini tanımlayan arayüz modeli. 4) Veri akışı gibi ilişkileri gösteren ilişki modelleri. 5) Alt sistemlerin bilgisayarlara dağılımını gösteren dağıtım modeli.

Q6. Describe the repository model. What are its advantages and disadvantages?
Repository modeli (s.247–249)

> *In the repository model, all shared data is held in a central database that all sub-systems can access. It is used in CASE toolsets, CAD and management information systems. Advantages: it is efficient for sharing large amounts of data, producers do not need to know how data is used, backup and security are centralised, and new tools are easy to integrate. Disadvantages: sub-systems must agree on a common data model, evolution is difficult, the same policy is forced on all sub-systems, and it is difficult to distribute.*

Repository modelinde bütün paylaşılan veri, bütün alt sistemlerin erişebildiği merkezi bir veritabanında tutulur. CASE araç takımları, CAD ve yönetim bilgi sistemlerinde kullanılır. Avantajlar: büyük veriyi paylaşmak verimli, üreten kullanımla ilgilenmez, yedekleme ve güvenlik merkezi, yeni araç eklemek kolay. Dezavantajlar: ortak veri modeli şart, evrim zor, herkese aynı politika, dağıtmak zor.

Q7. Describe the client-server model. What are its advantages and disadvantages?
Client-server modeli (s.249–250)

> *The system is organised as a set of servers that offer services, a set of clients that call these services, and a network. Clients use a request-reply protocol such as http. The main advantage is that it is a distributed architecture: it is easy to add new servers and upgrade them transparently. The disadvantages are that there may be no shared data model across servers, and changes to clients and servers may be required to get the full benefit of a new server.*

Sistem; hizmet sunan sunucular, bu hizmetleri çağıran istemciler ve bir ağ olarak düzenlenir. İstemciler http gibi istek-cevap protokolü kullanır. Ana avantajı dağıtık bir mimari olmasıdır: yeni sunucu eklemek ve şeffaf yükseltmek kolaydır. Dezavantajları: sunucular arasında ortak veri modeli olmayabilir ve yeni sunucunun faydası için istemci ve sunucularda değişiklik gerekebilir.

Q8. Describe the layered (abstract machine) model. What are its advantages and disadvantages?

Katmanlı model (s.250–252)

> *The system is organised into layers, each providing a set of services and acting as an abstract machine for the layer above. Examples are the OSI model and a version management system (configuration management, object management, database, operating system). Advantages: it supports incremental development, a layer can be replaced if its interface stays the same, changes affect only adjacent layers, and it is portable. Disadvantages: it can be difficult to structure systems this way, outer layers may need to punch through to inner layers, and performance can suffer because of multiple levels of interpretation.*

Sistem, her biri hizmet sunan ve üstündeki katman için soyut makine gibi davranan katmanlara ayrılır. Örnekler OSI modeli ve sürüm yönetim sistemidir (konfigürasyon, nesne yönetimi, veritabanı, işletim sistemi). Avantajlar: artımlı geliştirme, arayüz aynıysa katman değiştirilebilir, değişiklik sadece komşu katmanları etkiler, taşınabilir. Dezavantajlar: bu şekilde yapılandırmak zor olabilir, dış katmanlar iç katmanlara delip geçmek zorunda kalabilir, çoklu yorumlama yüzünden performans düşebilir.

Q9. Compare object-oriented decomposition and function-oriented pipelining.

OO ve pipeline karşılaştırması (s.252–255)

> *In object-oriented decomposition, the system is a set of loosely coupled objects with private state and operations. It is understandable and reusable, but objects must know each other's names and interfaces. In function-oriented pipelining, the system is a set of transformations that process input data into output data, like Unix pipes and filters. It supports reuse of transformations and is intuitive, but needs a common data format and is not suitable for interactive systems. The object model is more abstract because it does not show the sequence of operations.*

Nesne yönelimli ayrıştırmada sistem, özel durumu ve işlemleri olan gevşek bağlı nesnelerdir. Anlaşılır ve yeniden kullanılabilir, ama nesneler birbirinin adını ve arayüzünü bilmelidir. Fonksiyon odaklı boru hattında sistem, girdiyi çıktıya dönüştüren dönüşümlerdir (Unix pipe and filter gibi). Dönüşümlerin yeniden kullanımını destekler ve sezgiseldir, ama ortak veri formatı ister ve etkileşimli sistemlere uygun değildir. Nesne modeli işlemlerin sırasını göstermediği için daha soyuttur.

Q10. Describe the centralised control models.
Merkezi kontrol modelleri (s.256–257)

> *In centralised control, one sub-system controls the others. 1) The call-return model is a top-down subroutine model: control starts at the top of a hierarchy and passes down through subroutine calls, then returns. It is only for sequential systems. It is easy to analyse but exceptions are awkward to handle. 2) The manager model is for concurrent systems: one component is a system manager that starts, stops and coordinates other processes, often by polling sensors in an event loop. It is used in soft real-time systems.*

Merkezi kontrolde bir alt sistem diğerlerini kontrol eder. 1) Call-return modeli yukarıdan aşağı alt program modelidir: kontrol hiyerarşinin tepesinden başlar, çağrılarla aşağı iner ve geri döner. Sadece sıralı sistemler içindir. Analiz etmesi kolay ama istisnaları ele almak zahmetlidir. 2) Manager modeli eşzamanlı sistemler içindir: bir bileşen diğer süreçleri başlatan, durduran ve koordine eden yöneticidir, genelde sensörleri bir olay döngüsünde yoklar. Yumuşak gerçek zamanlı sistemlerde kullanılır.

Q11. Describe the event-driven control models.
Olay güdümlü kontrol modelleri (s.258–260)

> *Event-driven models are driven by externally generated events. 1) In broadcast models, sub-systems register an interest in events and an event and message handler passes the events to them. Evolution is simple and sub-systems do not need to know each other's names or locations, but a sub-system does not know if or when its events will be handled. 2) In interrupt-driven models, used in real-time systems, interrupts are detected by an interrupt handler and control is transferred immediately. They give very fast responses but are complex to program and difficult to validate.*

Olay güdümlü modeller dış olaylarla yönlendirilir. 1) Yayın modellerinde alt sistemler olaylara ilgi kaydeder ve olay ve mesaj işleyici olayları onlara iletir. Evrim basittir, alt sistemler birbirinin adını ya da yerini bilmez, ama bir alt sistem olayının işlenip işlenmeyeceğini ve ne zaman işleneceğini bilmez. 2) Gerçek zamanlı sistemlerde kullanılan kesme güdümlü modellerde kesmeler işleyici tarafından algılanır ve kontrol hemen aktarılır. Çok hızlı yanıt verir ama programlaması karmaşık ve doğrulaması zordur.

Q12. Suggest an appropriate control model for: a batch payroll system, tools from different vendors that must work together, a TV controller. (Ex. 11.7)
Kontrol modeli seçimi

> *Batch payroll system: call-return model, because it is a sequential system with no external events. Tools from different vendors: broadcast model, because tools can register for events and do not need to know each other's names or locations. TV controller: interrupt-driven model, because it must respond quickly to signals from the remote control.*

Toplu maaş bordrosu: call-return, çünkü dış olayı olmayan sıralı bir sistemdir. Farklı firmaların araçları: broadcast, çünkü araçlar olaylara kaydolur ve birbirinin adını ve yerini bilmesi gerekmez. TV denetleyicisi: interrupt-driven, çünkü kumandadan gelen sinyallere hızlı yanıt vermelidir.

Q13. What is the difference between generic models and reference models? Describe the OSI model.

Generic ve reference model, OSI (s.260–262)

> *Generic models are abstractions from real systems and can be reused directly in a design. Reference models are more abstract and idealised. They describe a larger class of systems and are used to communicate domain concepts and to compare and evaluate architectures. The OSI model is a seven-layer reference model for open systems interconnection: physical, data link, network, transport, session, presentation and application. In practice, performance problems mean that layers are often bypassed, so transparent replacement of a layer is hardly ever possible.*

Genel modeller gerçek sistemlerden soyutlamalardır ve tasarımda doğrudan yeniden kullanılabilir. Referans modeller daha soyut ve idealdir, daha geniş bir sistem sınıfını tanımlar ve alan kavramlarını iletmek, mimarileri karşılaştırmak ve değerlendirmek için kullanılır. OSI, açık sistemler arası bağlantı için 7 katmanlı bir referans modeldir: fiziksel, veri bağı, ağ, taşıma, oturum, sunum ve uygulama. Pratikte performans sorunları yüzünden katmanlar çoğu zaman atlanır, bu yüzden bir katmanın şeffaf değiştirilmesi neredeyse hiç mümkün olmaz.

Q14. What services does the ECMA reference model for CASE environments define? How can it be used? (Ex. 11.9)

ECMA CASE modeli (s.262–264)

> *It defines five services: data repository services, data integration services, task management services, message services and user interface services. It should also provide plug-in facilities for CASE tools. To compare two CASE toolsets, we can ask for each toolset how it provides each of these services, for example "how are data repository services provided?" and "does it provide task management?".*

Beş hizmet tanımlar: veri deposu, veri entegrasyonu, görev yönetimi, mesaj ve kullanıcı arayüzü hizmetleri. CASE araçları için eklenti olanağı da sağlamalıdır. İki CASE ürününü karşılaştırmak için her biri için bu hizmetleri nasıl sağladığını sorarız, örneğin "veri deposu hizmetleri nasıl sağlanıyor?" ve "görev yönetimi var mı?".

E Mini test

cevaplar en altta

1. The output of the architectural design process is a description of the:
   a) test plan
   b) software architecture
   c) user interface
   d) project schedule
2. If performance is critical, the architecture should use:
   a) many fine-grain components
   b) a few large-grain components
   c) shared data structures only
   d) many layers
3. If security is critical, the architecture should use:
   a) a layered structure with critical assets in the innermost layers
   b) a broadcast model
   c) redundant components only
   d) no layers

4. If maintainability is critical, you should use:
   a) large-grain components
   b) shared data structures
   c) fine-grain, self-contained components
   d) one big module
5. A pattern of system organisation such as client-server is an:
   a) architectural style
   b) ADL
   c) interrupt
   d) reference model
6. The basic elements of ADLs are:
   a) classes and objects
   b) components and connectors
   c) layers and pipes
   d) events and handlers
7. All shared data held in a central database is the:
   a) client-server model
   b) repository model
   c) layered model
   d) pipeline model
8. A repository that triggers sub-systems when data becomes available is a:
   a) blackboard model
   b) call-return model
   c) OSI model
   d) batch model
9. In the client-server model, servers:
   a) must know all clients
   b) need not know the identity or number of clients
   c) call the clients
   d) cannot be distributed
0. "Punching through" adjacent layers is a disadvantage of the:
   a) repository model
   b) layered model
   c) broadcast model
   d) object model
1. Unix pipes and filters are an example of:
   a) object-oriented decomposition
   b) function-oriented pipelining
   c) interrupt-driven control
   d) repository model
2. A main problem of the pipeline model is:
   a) objects must know each other's names
   b) a common data format is needed
   c) too fast responses
   d) no reuse
3. The call-return model is applicable only to:
   a) sequential systems
   b) concurrent systems
   c) distributed systems
   d) real-time systems
4. A central controller that polls sensors in a loop uses the:
   a) broadcast model
   b) manager (event-loop) model
   c) pipeline model
   d) repository model
5. In the broadcast model, a sub-system that generates an event:
   a) knows exactly who will handle it
   b) does not know if or when it will be handled
   c) must handle it itself
   d) stops all sub-systems
6. An airbag control system should use:
   a) call-return control
   b) interrupt-driven control
   c) batch sequential
   d) repository model

7. Models used mainly to compare and evaluate architectures are:

a) generic models
b) reference models
c) dynamic models
d) distribution models

8. How many layers does the OSI reference model have?

a) 4
b) 5
c) 7
d) 9

Cevap anahtarı: 1-b · 2-b · 3-a · 4-c · 5-a · 6-b · 7-b · 8-a · 9-b · 10-b · 11-b · 12-b · 13-a · 14-b · 15-b · 16-b · 17-b · 18-c